

Universally Composable Relaxed Asymmetric Password-Authenticated Key Exchange[★]

Shuya Hanai^[0009-0002-0738-4679], Keisuke Tanaka^[0000-0003-1330-4495],
Masayuki Tezuka^[0009-0003-5260-3372], and Yusuke Yoshida^[0000-0003-0905-5641]

Institute of Science Tokyo, Tokyo, Japan
`hanai.s.ac@m.titech.ac.jp`, `keisuke@is.titech.ac.jp`,
`tezuka.m.ac@m.titech.ac.jp`, `yoshida.yusuke@c.titech.ac.jp`

Abstract. Password-Authenticated Key Exchange (PAKE) establishes a secure channel between two parties who share a password. Asymmetric PAKE is a variant of PAKE, where one party stores a hash of the password to preserve security under the situation that the party is compromised. The security of PAKE and asymmetric PAKE is often analyzed in the framework of universal composability (UC).

Abdalla et al. (CRYPTO '20) relaxed the UC security of PAKE and showed that the relaxed security still guarantees reasonable properties. This relaxation makes it possible to prove the security in the UC framework for several PAKE protocols.

In this paper, we propose a relaxed functionality of asymmetric PAKE by following the approach of Abdalla et al. We prove that the SPAKE2+ protocol UC-realizes this functionality. We also define a more relaxed functionality and prove that a variant of the AuCPace protocol UC-realizes it.

Keywords: Asymmetric Password-Authenticated Key Exchange, Universal Composability

1 Introduction

1.1 Background

PAKE and aPAKE. Password-Authenticated Key Exchange (PAKE) was proposed by Bellare and Merritt [7] aiming to realize an authentication using the passwords of two parties. PAKE is often called symmetric PAKE or balanced PAKE. PAKE protocols allow two parties who share one password to share one session key. The password is supposed to be simple and human-memorable.

When a password used in a PAKE protocol is stolen by an adversary, the authentication of the PAKE protocol is trivially broken. To reduce this risk,

[★] A part of this work was supported by JSPS KAKENHI JP24H00071, JP23K16841 and JST CREST JPMJCR2113. This paper is the full version of the paper presented in SCN2024 [23]. In this paper, we add some discussions, including Section 7 describing the variants of the functionality we defined and the appendixes showing the full proofs. We also modify some errors found in the conference version.

Bellovin and Merritt [8] proposed asymmetric PAKE (aPAKE). aPAKE is often called augmented PAKE or verifier-based PAKE. Typically, aPAKE captures a situation where a client has a password and a server stores a password file. The password file is a hashed password, and the server does not know the password in cleartext. This mechanism prevents an adversary that compromises the server and steals the password file from impersonating the client.

Relaxation of UC PAKE. Security definitions of PAKE are formalized in a game-based model or a simulation-based model. Bellare, Pointcheval, and Rogaway [6] proposed a game-based security definition of PAKE, which is called the BPR model. After their work, Canetti, Halevi, Katz, Lindell, and MacKenzie [13] introduced a simulation-based security definition. This definition uses the universal composability (UC) framework proposed by Canetti [11,10].

Several PAKE protocols are not proven secure in the UC framework but in the game-based model. To clarify and overcome this gap, Abdalla, Barbosa, Bradley, Jarecki, Katz, and Xu [1,2] proposed two relaxed security definitions of PAKE in the UC framework; lazy-extraction PAKE (lePAKE) and relaxed PAKE (rPAKE).

The functionality of lePAKE allows an adversary in a session to make a password guess after the session is completed and know a session key if the guess is correct. This relaxation makes it possible to prove the security of some PAKE protocols in the UC framework. In the work of Abdalla et al. [1,2], they proved the security of SPAKE2 [5], SPEKE [28], and TBPEKE [33]. Furthermore, the lePAKE functionality was used to prove the security of 2DH-SPAKE2 [31,32], 2DH-PPK [31,32], and CPace [4,21,22].

The functionality of rPAKE also allows an adversary in a session to make a password guess after the session is completed, but the adversary can only know whether the guess is correct.

Abdalla et al. [1,2] showed that lePAKE satisfies weak forward secrecy, which means that session keys in sessions not interfered with by the adversary are secure even if the adversary gets passwords of these sessions. They also showed that rPAKE satisfies perfect forward secrecy, which means that session keys are secure if the adversary finds the passwords used in the sessions after the sessions are completed.

Abdalla et al. [1,2] presented that the security of lePAKE can be enhanced to the security of rPAKE with explicit authentication (rPAKE-EA) by adding key confirmation to lePAKE. So, lePAKE protocols with key confirmation achieve perfect forward secrecy.

As another relaxation method, Abdalla, Barbosa, Katz, Loss, and Xu [3] formalized the algebraic group model in the UC framework. They also proved that SPAKE2 and CPace realize the original PAKE functionality in the model.

Security of aPAKE. The security of aPAKE was studied in two models as well as PAKE. Gentry, MacKenzie, and Ramzan [17] proposed an ideal functionality of aPAKE in the UC framework. This functionality and its variants

have been used to provide security proofs for various aPAKE protocols such as [19,25,24,21,22,36,34,31,32,27,29,30]. In the game-based model, some aPAKE protocols were proven secure, such as [33,35,9].

1.2 Motivations

(1) Relaxation of UC aPAKE. Some aPAKE protocols require a relaxation method for UC aPAKE to prove their security in the UC framework as in the case of PAKE. However, attempts have not been well made to define the aPAKE functionality relaxed in the same way as lePAKE. Haase and Labrique [21,22] proposed a relaxed aPAKE model in the UC framework based on a similar technique to defining lePAKE. But, Abdalla et al. [1,2] pointed out that the relaxed aPAKE model partially does not capture the security of aPAKE precisely. The first problem pointed out is that an adversary in the model can make a password guess against a session without actively interfering with the session. The second problem is that the timing for an adversary making its password guess for one party after the session is completed is restricted to before the other party ends its session. Therefore, it is desirable to redefine the relaxed aPAKE model precisely.

(2) SPAKE2+. Cash, Kiltz, and Shoup [16] proposed the SPAKE2+ protocol. This is one of the aPAKE protocols whose security should be considered using the relaxed model. SPAKE2+ has a simple structure and the RFC9383 document describing SPAKE2+ was published recently [37]. SPAKE2+ has a similar structure to the PAKE protocol SPAKE2, which was proven to UC-realize the lePAKE functionality by Abdalla et al. [1,2]. Shoup [36] pointed out that SPAKE2+ does not meet the original aPAKE functionality for the same reason that SPAKE2 does not meet the original PAKE functionality. Shoup [36] also showed the variants of SPAKE2+ with explicit authentication and proved them secure using a relaxed aPAKE functionality. However, the proof of security of SPAKE2+ without key confirmation has not been provided in the UC framework to our knowledge.

(3) le-AuCPace. Another aPAKE protocol considered in this paper is a compiler from lePAKE to aPAKE which was proposed by Haase and Labrique [21,22]. We refer to this compiler as **le-AuCPace** in this paper. In the proposal, the authors mainly focused on the aPAKE protocol AuCPace, and le-AuCPace was presented as an AuCPace variant without explicit key confirmation. They suggested that le-AuCPace satisfies the aPAKE functionality relaxed in the same way as lePAKE. Later, Haase [20] tried to prove the security of le-AuCPace as a response to the requests presented in the Crypto Forum Research Group PAKE selection process.

(4) Relationship between relaxed aPAKE models. As a further problem, there is no formalization of a compiler dealing with aPAKE functionalities relaxed in the two directions to our knowledge. An aPAKE model relaxed in the

same way as lePAKE is used in [21,22] and aPAKE models relaxed in the same way as rPAKE are used in [29,30,36]. However, each of those studies did not address clarifying the relationship between those two relaxed models as was done for the relaxed functionalities of PAKE by Abdalla et al. [1,2].

1.3 Our Contributions

In this paper, we define some relaxed functionalities of aPAKE based on the method used to PAKE by Abdalla et al. [1,2] and use these functionalities to prove the security of two aPAKE protocols. We describe our contributions numbering them as well as the motivations.

(1) Defining relaxed functionalities of aPAKE. In Section 3, we formalize the relaxed functionalities of aPAKE.

- We define the lazy-extraction aPAKE (le-aPAKE) functionality $\mathcal{F}_{\text{le-aPAKE}}$ by applying the relaxation method of lePAKE to the UC security definition of aPAKE. Compared to the relaxed functionality of aPAKE in the prior study [21,22], $\mathcal{F}_{\text{le-aPAKE}}$ does not have the problems pointed out in the functionality.
- We define the functionality of weak le-aPAKE $\mathcal{F}_{\text{w-le-aPAKE}}$ to capture the security of le-AuCPace. $\mathcal{F}_{\text{w-le-aPAKE}}$ is a weaker variant of $\mathcal{F}_{\text{le-aPAKE}}$ in that $\mathcal{F}_{\text{w-le-aPAKE}}$ allows an adversary attacking a session of a w-le-aPAKE protocol to successfully impersonate the server to the client if the adversary steals the password file of the server even after the session is completed. We illustrate the detail of the motivation for this relaxation in Section 5.
- We define the relaxed aPAKE (r-aPAKE) functionality $\mathcal{F}_{\text{r-aPAKE}}$ by applying the relaxation method of rPAKE to aPAKE.
- We define the r-aPAKE with explicit authentication (r-aPAKE-EA) functionality $\mathcal{F}_{\text{r-aPAKE-EA}}$ as a natural extension of $\mathcal{F}_{\text{r-aPAKE}}$.

In combination with the aforementioned relaxations, our formalized functionalities are additionally relaxed to allow the adversary to obtain information on whether the password of the client matches the password of the server. This relaxation allows the adversary to determine whether an attack using the stolen password file will be successful before launching the attack. This relaxation was already incorporated in [36,31,32], and $\mathcal{F}_{\text{r-aPAKE-EA}}$ in this paper seems to be similar to the relaxed aPAKE functionality defined in [36].

(2) SPAKE2+ UC-realizes $\mathcal{F}_{\text{le-aPAKE}}$. In Section 4, we prove that the aPAKE protocol SPAKE2+ satisfies the functionality of le-aPAKE. In this proof, we need to show that if an adversary does not know the password file when the adversary interferes with the session, then the session key of the client is secure. To show this, we use an assumption introduced in the proof of perfect forward secrecy of SPAKE2 in [1,2]. We also explain that SPAKE2+ does not UC-realize the original aPAKE functionality and requires the relaxation of the le-aPAKE functionality to prove its security.

(3) le-AuCPace UC-realizes $\mathcal{F}_{w\text{-le-aPAKE}}$. In Section 5, we prove that le-AuCPace satisfies the functionality of w-le-aPAKE. We observe that le-AuCPace does not realize $\mathcal{F}_{\text{le-aPAKE}}$, which is realized by SPAKE2+. When an adversary actively attacks a session of le-AuCPace and steals the password file, the session key of the client is insecure even if the timing of this password stealing is after the session is completed. Therefore, le-AuCPace only satisfies $\mathcal{F}_{w\text{-le-aPAKE}}$. This is because the lePAKE protocol, which is used in le-AuCPace, only satisfies weak forward secrecy. When an adversary attacks SPAKE2+ in the same way, the adversary cannot get the session key because SPAKE2+ has a common structure with SPAKE2, which was shown to satisfy perfect forward secrecy in [1,2]. We illustrate this security feature of le-AuCPace in detail in this section.

(4) Compiler from w-le-aPAKE or le-aPAKE to r-aPAKE-EA. In Section 6, we show a compiler from w-le-aPAKE or le-aPAKE to r-aPAKE-EA. The structure of this compiler is based on the compiler from lePAKE to rPAKE-EA presented in [1,2]. The compiler we propose can be explained as a w-le-aPAKE protocol or a le-aPAKE protocol with key confirmation.

Combining this compiler with our contributions mentioned above, we can re-examine the results of several security proofs in some previous studies. When considered with our contribution (1) and this compiler, our result can be interpreted as an intuitive revalidation of the work by Shoup [36], which states that two variants of SPAKE2+ with explicit authentication UC-realize a variant of r-aPAKE-EA functionality. The result in (3) and this compiler intuitively support the statement by Haase and Labrique [21,22] that the aPAKE protocol AuCPace, which is defined as le-AuCPace with key confirmation, is secure.

(4+) Variations of r-aPAKE-EA. We observe that w-le-aPAKE with key confirmation satisfies two variants of r-aPAKE-EA and the two variants capture stronger security than r-aPAKE-EA. In Section 7, we propose the two variants and investigate their features.

The two variants and r-aPAKE-EA differ in terms of how to allow the adversary to know the information on whether the password of a client matches the password of a server. r-aPAKE-EA gives the information to the adversary whenever clients are initiated. The first variant is an ideal functionality that gives that information conditionally. The second variant is an ideal functionality that does not give that information to the adversary, but w-le-aPAKE satisfies this variant by restricting the client to wait for a key confirmation message from the server before sending it. To consider these variants, we change w-le-aPAKE in this discussion not to tell the adversary whether a client and server use a common password, and this change does not affect the realizability of le-AuCPace to w-le-aPAKE. Considering the ease of constructing the simulator, r-aPAKE-EA has the weakest security and the second variant has the strongest security of the three functionalities.

We discuss three points about r-aPAKE-EA and the two variants. First, we discuss that when aPAKE is regarded as a tool to realize another functionality,

incorporating r-aPAKE-EA has no effect in many cases on the functionality by an argument of Shoup [36]. Second, we discuss when there is a difference between r-aPAKE-EA and the first variant, and we confirm that the first variant can capture security features in w-le-aPAKE with key confirmation not considered by r-aPAKE-EA. Third, the restriction for key confirmation in the second variant seems to be less efficient than sending key confirmation messages asynchronously, but we explain that the number of rounds on this restriction is as efficient as asynchronous key confirmation if the server sends the key confirmation message at the same time as a last message for key exchange.

2 Preliminaries

Notation. For a finite space S and an element $x \in S$, we use a notation $x \leftarrow_R S$ to denote that x is sampled uniformly at random from S . The protocols in this paper use a cyclic group $\mathbb{G}$ of prime order p . A generator of the group $\mathbb{G}$ is denoted by $g \in \mathbb{G}$.

Assumptions. We introduce the computational Diffie-Hellman (CDH) assumption and the gap CDH (GCDH) assumption to show the security of SPAKE2+ and le-AuCPace. We follow how to express assumptions in [2].

The CDH assumption states that given g^{a_1}, g^{a_2} for $a_1, a_2 \leftarrow_R \mathbb{Z}_p$, it is hard to compute $g^{a_1 a_2}$. The GCDH assumption states that the CDH assumption holds even if the decision oracle $\text{DDH}(\cdot, \cdot, \cdot)$ is available, where $\text{DDH}(g^{a_1}, g^{a_2}, g^z)$ returns 1 if $g^{a_1 a_2} = g^z$ holds and 0 otherwise. Formally, the GCDH assumption is defined as follows.

Definition 1. (*The Gap CDH (GCDH) assumption. This expression is based on [2] Definition 5.1.*).

The GCDH assumption states that for any probabilistic polynomial-time adversary $\mathcal{A}$, the following advantage probability is negligible.

$$\text{Adv}_{\mathcal{A}}^{\text{GCDH}} = \Pr [C = g^{a_1 a_2} | a_1, a_2 \leftarrow_R \mathbb{Z}_p; C \leftarrow \mathcal{A}^{\text{DDH}(\cdot, \cdot, \cdot)}(g^{a_1}, g^{a_2})]$$

For the security proof of SPAKE2+, we show the Gap Discrete Logarithm (GDL) problem and the Gap discrete logarithm Password-based Chosen-basis Computational Diffie-Hellman (GPCCDH) assumption.

Definition 2. (*The Gap Discrete Logarithm (GDL) assumption. This expression is based on [2] Definition 5.2.*).

The GDL assumption states that for any probabilistic polynomial-time adversary $\mathcal{A}$, the following advantage probability is negligible.

$$\text{Adv}_{\mathcal{A}}^{\text{GDL}} = \Pr [c = a | a \leftarrow_R \mathbb{Z}_p; c \leftarrow \mathcal{A}^{\text{DDH}(\cdot, \cdot, \cdot)}(g^w)]$$

We consider that the GDL assumption holds when we assume the GCDH assumption in this paper.

The GPCCDH assumption was introduced by Abdalla et al. [2] to prove the perfect forward secrecy of SPAKE2.

Definition 3. (*The Gap discrete logarithm Password-based Chosen-basis Computational Diffie-Hellman (GPCCDH) assumption. A modified version of [2] Definition 5.3).*

The GPCCDH assumption states that for any probabilistic polynomial-time adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, the following advantage probability is negligible.

$$\text{Adv}_{\mathcal{A}}^{\text{GPCCDH}} = \Pr \left[\begin{array}{c} C = \text{CDH}(A_1/B_1^{pw}, \\ A_2/B_2^{pw}) \end{array} \middle| \begin{array}{c} a_1, b_1, b_2 \leftarrow_{\mathbb{R}} \mathbb{Z}_p \\ A_1 \leftarrow g^{a_1}; B_1 \leftarrow g^{b_1}; B_2 \leftarrow g^{b_2} \\ (A_2, st) \leftarrow \mathcal{A}_1^{\text{DDH}(\cdot, \cdot, \cdot)}(B_1, B_2, A_1) \\ pw \leftarrow_{\mathbb{R}} \mathcal{P}; C \leftarrow \mathcal{A}_2^{\text{DDH}(\cdot, \cdot, \cdot)}(pw, st) \end{array} \right]$$

$\mathcal{P}$ represents a password space. In the definition of the GPCCDH assumption in [2], the distribution where b_2 is sampled is defined as dependent on b_1 , but we simply define the assumption as above.

The PCCDH assumption, which equals the GPCCDH assumption that the adversary cannot access the DDH oracle, was originally defined by Abdalla and Pointcheval [5] to prove the security of SPAKE2 in the game-based model. They also presented two lemmas to show the relationship between the PCCDH assumption and the CDH assumption. The first is a lemma that can be used when the password space is small, and the lemma shows that even in such case, the difficulty of solving the PCCDH assumption can be reduced to the difficulty of solving the CDH problem ([5] Lemma 3.4). This lemma is extended by Abdalla et al. [2] as the relationship between the GPCCDH assumption and GCDH assumption. The second lemma shows that the PCCDH assumption can be reduced to the CDH problem when the password space is large ([5] Lemma 3.5).

The proof of the second lemma is similar to the proof of the first lemma. So, we extend the second lemma to the gap version and use this as the relationship between the GPCCDH assumption and the GCDH assumption as did for the first lemma in [2].

Lemma 1. (*The gap version of [5] Lemma 3.5).*

For all GPCCDH adversaries $\mathcal{A}$, there exists a GCDH adversary $\mathcal{B}$ that satisfies the following:

$$\text{Adv}_{\mathcal{A}}^{\text{GPCCDH}} \geq \epsilon \geq \frac{8}{|\mathcal{P}|} \implies \text{Adv}_{\mathcal{B}}^{\text{GCDH}} \geq \frac{\epsilon^2}{32}$$

Based on this lemma, we consider that the GPCCDH assumption holds when we assume the GCDH assumption and the space $\mathcal{P}$ is large in this paper.

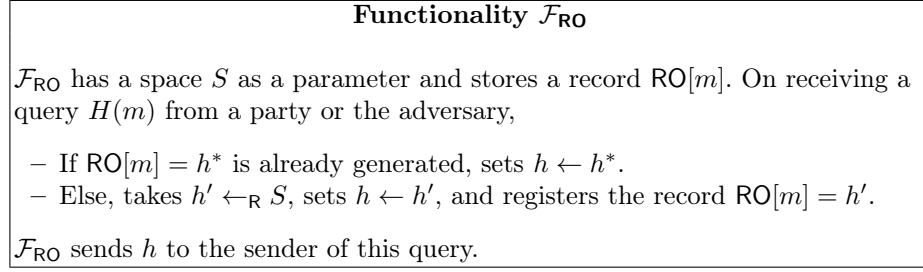


Fig. 1. The random oracle functionality $\mathcal{F}_{\text{RO}}$.

2.1 Models in UC Framework

We use the Random Oracle (RO) model and the Common Reference String (CRS) model in the SPAKE2+ protocol and the le-AuCPace protocol. These models are considered functionalities in the UC framework and the simulator in the ideal world simulates the behavior of the functionalities.

Random oracle model. Figure 1 shows the definition of the functionality of the random oracle $\mathcal{F}_{\text{RO}}$. Some works defined $\mathcal{F}_{\text{RO}}$ similarly, for example [32,24,26]. In the protocols in this paper, we model hash functions as random oracles. The value ranges of random oracles follow the specifications of protocols.

Common reference string model. Figure 2 gives the definition of the common reference string functionality. Some works defined this functionality or its variant similarly, such as [11,14,15,12].

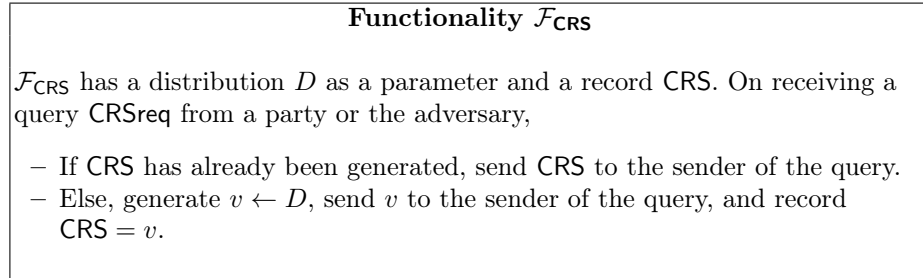


Fig. 2. The common reference string functionality $\mathcal{F}_{\text{CRS}}$.

2.2 Definitions of $\mathcal{F}_{\text{lePAKE}}$

In Figures 3, we show the definition of the ideal functionality of lePAKE $\mathcal{F}_{\text{lePAKE}}$, which is used as a part of le-AuCPace defined in Section 5. This definition is

Functionality $\mathcal{F}_{\text{lePAKE}}$	
<u>Password authentication</u>	
– On $(\text{NewSession}, sid, P, P', pw, \text{role})$ from P, send $(\text{NewSession}, sid, P, P', \text{role})$ to $\mathcal{S}$. If this is the first NewSession message for sid, record $\langle sid, P, P', pw, \text{clt} \rangle$ and mark it fresh.	
<u>Active session attack</u>	
– On $(\text{TestPwd}, sid, P, pw^*)$ from $\mathcal{S}$, if there is a record $\langle sid, P, P', pw, \text{role} \rangle$ marked fresh, then do: • If $pw = pw^*$, mark the record compromised, and send “correct guess” to $\mathcal{S}$. • Otherwise, mark the record interrupted, and send “wrong guess” to $\mathcal{S}$.	
<u>Active session attack (for relaxation)</u>	
– On $(\text{RegisterTest}, sid, P)$ from $\mathcal{S}$, if there is a record $\langle sid, P, *, *, * \rangle$ marked fresh, mark the record interrupted, and flag it tested. – On $(\text{LateTestPwd}, sid, P, pw^*)$ from $\mathcal{S}$, if there is a record $\langle sid, P, P', pw, \text{role}, K \rangle$ marked completed and flagged tested, remove the flag from the record and do: • If $pw^* = pw$, then send K to $\mathcal{S}$. • Otherwise, take K' uniformly at random from the key space and send K'.	
<u>Key generation</u>	
– On $(\text{NewKey}, sid, P, k^*)$ from $\mathcal{S}$, if there is a record $\langle sid, P, P', pw, \text{role} \rangle$ not marked completed, then do: • If the record is marked compromised, or either P or P' is corrupted, then $k \leftarrow k^*$. • Else if the record is marked fresh and there is a completed record $\langle sid, P', P, pw, \text{rôle}, \hat{k} \rangle$ that satisfies $\text{role} = \text{rôle}$, and that was fresh when P' outputted $(sid, \hat{k})$, then $k \leftarrow \hat{k}$. • Otherwise, set $k \leftarrow_{\mathcal{R}} \{0, 1\}^{\kappa}$. Overwrite the record with $\langle sid, P, P', pw, \text{role}, k \rangle$, mark it completed, and output (sid, k) to P.	

Fig. 3. $\mathcal{F}_{\text{lePAKE}}$. This definition is based on the definition of the lePAKE functionality in [1,2], and we make some modifications to the definition to suit the style of this paper.

based on [1,2] and we make subtle adjustments to this definition to suit the discussion of this paper.

3 Relaxations of UC aPAKE

In this section, we define the four functionalities, the le-aPAKE functionality $\mathcal{F}_{\text{le-aPAKE}}$, the weak le-aPAKE functionality $\mathcal{F}_{\text{w-le-aPAKE}}$, the r-aPAKE functionality $\mathcal{F}_{\text{r-aPAKE}}$, and the r-aPAKE-EA functionality $\mathcal{F}_{\text{r-aPAKE-EA}}$.

Figures 4 and 5 provide the descriptions of $\mathcal{F}_{\text{le-aPAKE}}$, $\mathcal{F}_{\text{w-le-aPAKE}}$, and $\mathcal{F}_{\text{r-aPAKE}}$. Figure 6 gives the part of “Key generation and explicit authentication” used in $\mathcal{F}_{\text{r-aPAKE-EA}}$. This part is from $\mathcal{F}_{\text{rPAKE-EA}}$ defined by Abdalla et al. [1,2] based on the study by Groce and Katz [18]. $\mathcal{F}_{\text{r-aPAKE-EA}}$ is defined as $\mathcal{F}_{\text{r-aPAKE}}$ removed the queries categorized as “Key generation” and added the queries in Figure 6. In the functionalities we propose, the usage of a session identifier *sid* and a user account identifier *uid* follows the aPAKE functionality used in [19,34].

The structures of the aPAKE functionalities we define are based on the original aPAKE functionality $\mathcal{F}_{\text{aPAKE}}$ defined by Gentry et al. [17]. The original aPAKE functionality and the functionalities we propose have the **TestPwd** interface to capture online dictionary attacks. They also have interfaces named **StorePwdFile**, **StealPwdFile**, **OfflineTestPwd**, and **Impersonate** to capture the security in the situation where servers are compromised. In the real world, if an adversary compromises a server and steals the password file, the adversary can successfully pretend the server to the client. To capture this attack, in the ideal world, an adversary that has sent a **StealPwdFile** query can choose the session key of the client using the **Impersonate** query. Also, in the real world, if an adversary steals a password file, the adversary can test whether some passwords chosen by the adversary match the stolen password file. In the ideal world, the adversary can verify its password guess by using the **OfflineTestPwd** query.

In the functionalities we propose, we make three changes to relax the original aPAKE functionality. First, we add two queries to $\mathcal{F}_{\text{aPAKE}}$, the **RegisterTest** query, and the **LateTestPwd** query, to allow an adversary to make a password guess after a session is completed.

Second, we relax the definition of the **CltSession** query, which is used to initiate clients. Because of this relaxation, the adversary in the ideal world knows whether the server and client in a session use the same password.

Third, we make changes to the structure of queries involving key generations on session completion. In this paper, we deal with two types of aPAKE, aPAKE with implicit authentication ($\mathcal{F}_{\text{le-aPAKE}}$, $\mathcal{F}_{\text{w-le-aPAKE}}$, and $\mathcal{F}_{\text{r-aPAKE}}$) and aPAKE with explicit authentication ($\mathcal{F}_{\text{r-aPAKE-EA}}$). We therefore take the structure concerning key generations from the PAKE functionalities defined by Abdalla et al. [1,2] and incorporate it into the functionalities we define.

RegisterTest and LateTestPwd. As in the functionalities of rPAKE and the lePAKE in [1,2], we append the **RegisterTest** query and the **LateTestPwd** query

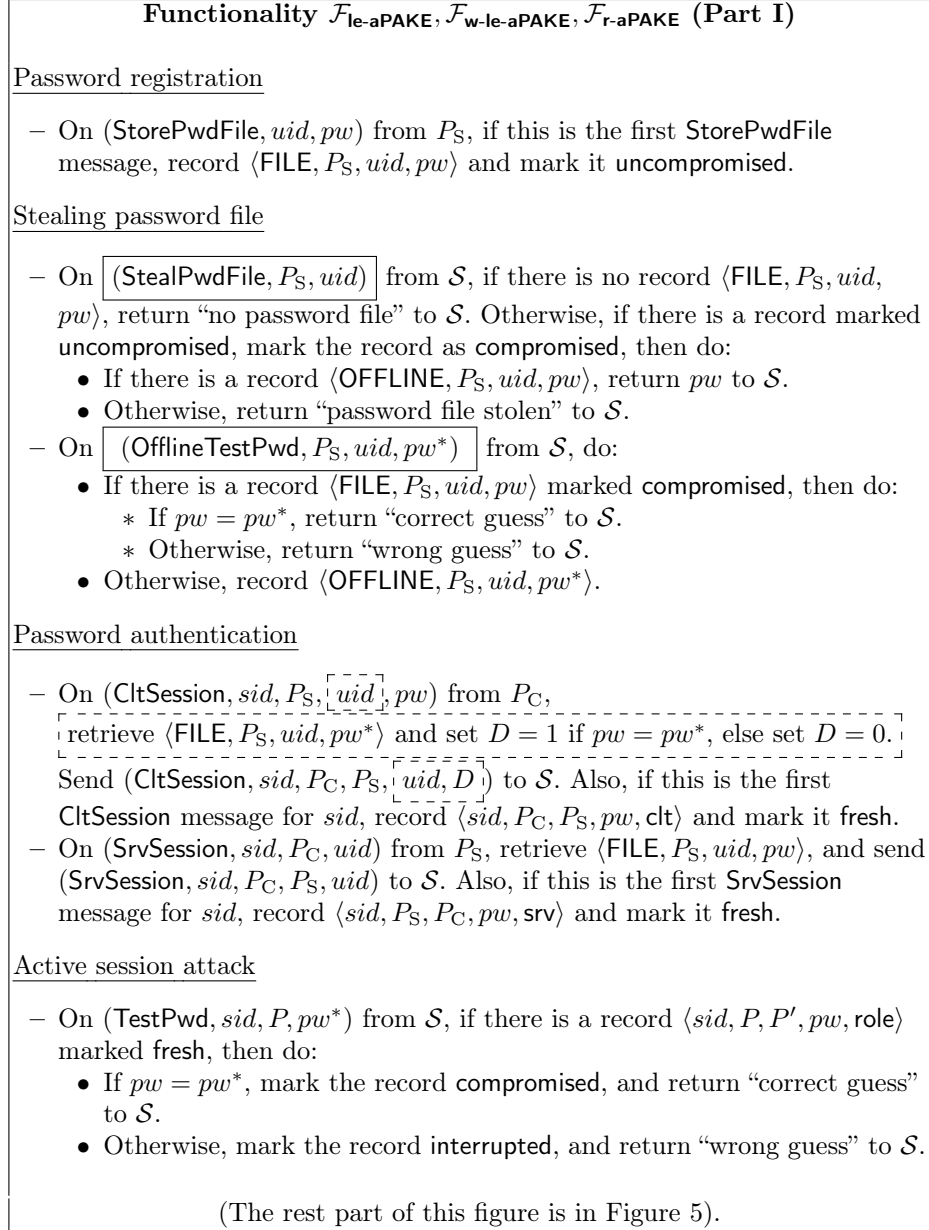


Fig. 4. $\mathcal{F}_{\text{le-aPAKE}}, \mathcal{F}_{\text{w-le-aPAKE}},$ and $\mathcal{F}_{\text{r-aPAKE}}$ (Part I). The queries surrounded by single boxes can only be used when the environment allows. In the original aPAKE functionality defined by Gentry et al. [17], the text in the dashed box is omitted. The description of the rest parts is in the caption of Figure 5.

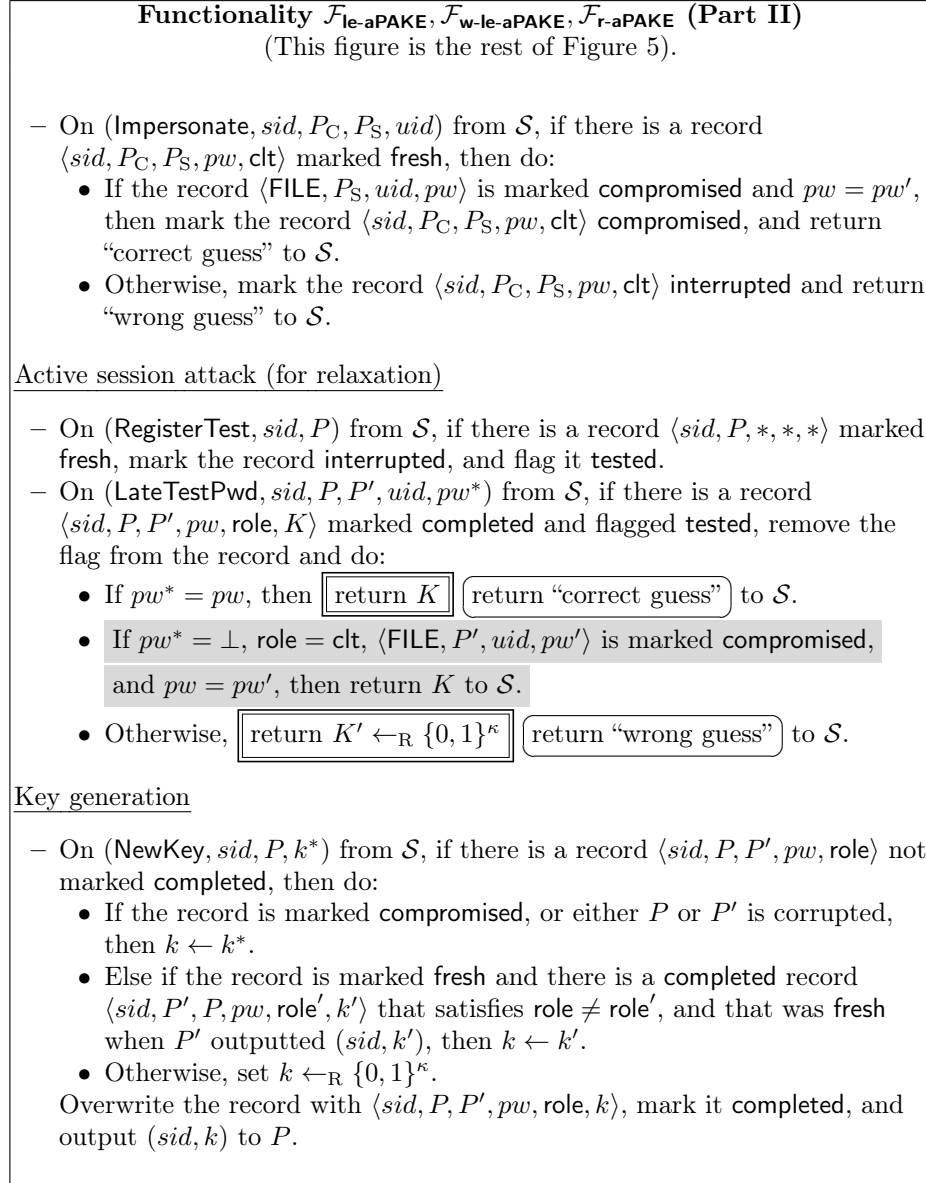


Fig. 5. $\mathcal{F}_{\text{le-aPAKE}}, \mathcal{F}_{\text{w-le-aPAKE}},$ and $\mathcal{F}_{\text{r-aPAKE}}$ (Part II). In the LateTestPwd query, the branches framed by double boxes are used when the ideal functionality is $\mathcal{F}_{\text{le-aPAKE}}$ and $\mathcal{F}_{\text{w-le-aPAKE}}$. The branches framed by rounded boxes are used when the ideal functionality is $\mathcal{F}_{\text{r-aPAKE}}$. The text in the grey boxes is only used by $\mathcal{F}_{\text{w-le-aPAKE}}$.

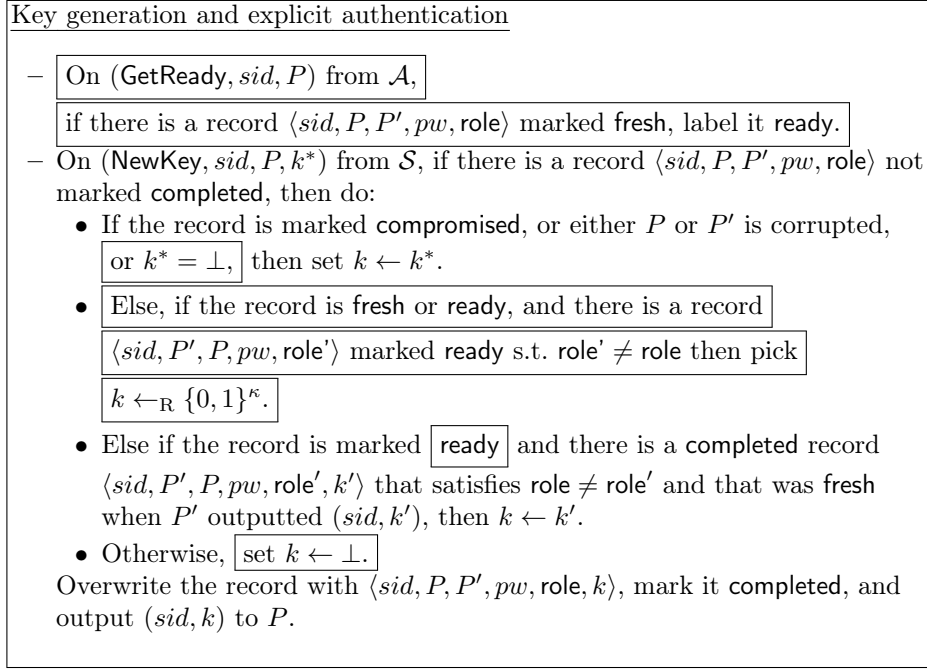


Fig. 6. The “Key generation and explicit authentication” part of r-aPAKE-EA functionality $\mathcal{F}_{r\text{-aPAKE-EA}}$. This figure is from the work of Abdalla et al. [1,2]. The text in the boxes highlights the differences from the functionalities that capture aPAKE protocols with implicit authentication.

into the functionalities we propose. The **RegisterTest** query allows the adversary in the ideal world to make their password guesses using the **LateTestPwd** query after the sessions are completed, instead of active session attacks using the **TestPwd** query or the **Impersonate** query before the sessions are completed. In $\mathcal{F}_{\text{le-aPAKE}}$ and $\mathcal{F}_{\text{w-le-aPAKE}}$, if the password guess is correct, these functionalities return the session key of the attacked party. Otherwise, they return a random value irrelevant to the session key, and this behavior makes the functionalities stronger than informing that the password guess is wrong. In $\mathcal{F}_{r\text{-aPAKE}}$ and $\mathcal{F}_{r\text{-aPAKE-EA}}$, these functionalities return whether the password guess is correct.

In $\mathcal{F}_{\text{w-le-aPAKE}}$, if the adversary steals the password file and the password of the client matches the password of the server, the adversary can obtain the session key by sending a **LateTestPwd** query with $\perp$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ after the session is completed. This change allows an active adversary to successfully impersonate the server, even if the adversary steals the password file after the session is completed. Due to this relaxation, $\mathcal{F}_{\text{w-le-aPAKE}}$ is weaker than $\mathcal{F}_{\text{le-aPAKE}}$.

ClSession. In the functionalities we propose, the value D is added to the `ClSession` message sent to $\mathcal{S}$. If the client and the server use the same password, $D = 1$, otherwise $D = 0$. D is not contained in the original aPAKE functionality.

This change is mainly needed for $\mathcal{F}_{\text{le-aPAKE}}$ to be used to prove the security of SPAKE2+. The value D enables the adversary that wants to find the session key of the client to determine whether an attack with the `Impersonate` query is successful before sending this query to the functionality. If the adversary steals the password file, the adversary can choose to make the `Impersonate` query or the `RegisterTest` query before the session is completed to attack the client side. In $\mathcal{F}_{\text{le-aPAKE}}$, the attack with the `Impersonate` query fails when the password of the client does not match the password of the server. On the other hand, the attack using the `RegisterTest` query fails when the adversary cannot guess the password of the client in the `LateTestPwd` query after the session is completed. Hence, the best strategy for the adversary that steals the password file before the session is completed is to use the `Impersonate` query if $D = 1$. Otherwise, to use the `RegisterTest` query.

We incorporate this relaxation not only in $\mathcal{F}_{\text{le-aPAKE}}$, but also $\mathcal{F}_{\text{w-le-aPAKE}}$, $\mathcal{F}_{\text{r-aPAKE}}$, and $\mathcal{F}_{\text{r-aPAKE-EA}}$. This is because when defining the compiler `w-le-aPAKE` or `le-aPAKE` to `r-aPAKE-EA`, it is appropriate to incorporate such relaxation in all these functionalities. In the discussion on $\mathcal{F}_{\text{w-le-aPAKE}}$ only, this relaxation is not necessary because the adversary has a similar capability to attack whether the session is completed or not.

This relaxation is already incorporated in the relaxed aPAKE functionality formalized by Shoup [36]. They seem to introduce this relaxation mainly to keep the logic of the simulation simple and discussed their reason for incorporating this relaxation in Remark 3 of their study. Liu, Liu, Han, and Gu [31,32] also incorporated this relaxation following the study by Shoup [36]. Because of this relaxation and the relaxation concerning active session attacks for completed sessions, the aPAKE functionality used in the study by Shoup [36] seems to be similar to $\mathcal{F}_{\text{r-aPAKE-EA}}$ formalized in this paper.

We note that, in our defined functionalities, the server must be initialized before the client is initiated because the aPAKE functionalities must see the password of the server when the client is initiated. This restriction of the order of initialization is explained in [36] Remark 6.

Implicit authentication and explicit authentication. We design the functionalities $\mathcal{F}_{\text{r-aPAKE}}$, $\mathcal{F}_{\text{le-aPAKE}}$, and $\mathcal{F}_{\text{w-le-aPAKE}}$ to capture the security of aPAKE protocols with implicit authentication. In those functionalities, we do not incorporate the `TestAbort` query, which captures explicit authentication in the original aPAKE functionality. So, the “key generation” parts in these functionalities are identical to the part of the same name in the functionalities for PAKE with implicit authentication in the study by Abdalla et al. [1,2].

For defining the functionality for aPAKE protocols with explicit authentication, there are some options. The original aPAKE functionality captures this authentication by using the `TestAbort` query. Hesse [24,25] pointed out a short-

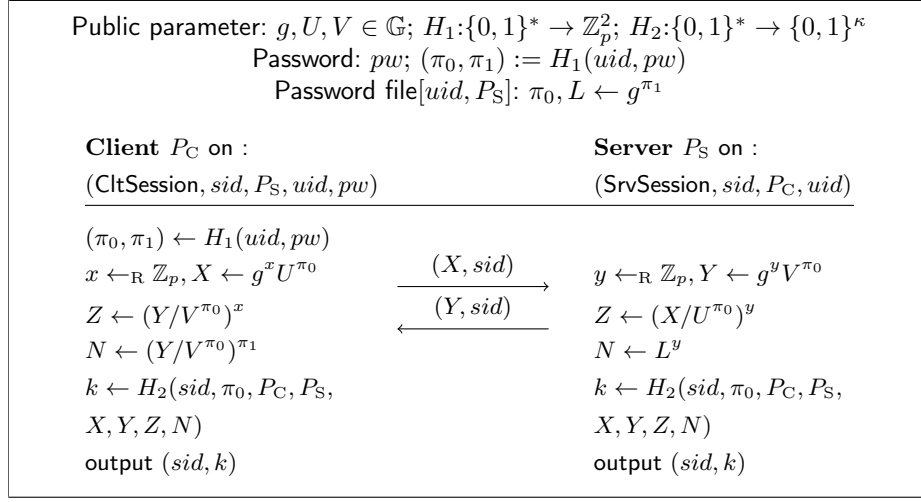


Fig. 7. The SPAKE2+ protocol [16].

coming in the mechanism of the query and re-modeled it as a modified **NewKey** query. As another option, in the study by Abdalla et al. [1,2], the functionality of rPAKE-EA $\mathcal{F}_{\text{rPAKE-EA}}$ captures explicit authentication by introducing a changed **NewKey** query and the **GetReady** query. The structure of these queries in the rPAKE-EA functionality is based on the study by Groce and Katz [18]. The method of Hesse and the method of Groce and Katz have in common that the ideal functionalities determine whether a session has matched passwords or unmatched passwords. Shoup [36] defines a functionality that allows the simulator to specify all the parties output the abort messages by modifying **CltSession** to make the simulator know whether a client and a server use a common password.

To keep consistency with the discussion by Abdalla et al. [1,2], we incorporate the **NewKey** query and the **GetReady** query in $\mathcal{F}_{\text{rPAKE-EA}}$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$. $\mathcal{F}_{\text{r-aPAKE-EA}}$ allows the simulator to judge whether a client and server have a matching password. The simulator can designate all the parties that output the abort messages like the method of Shoup [36]. However, $\mathcal{F}_{\text{r-aPAKE-EA}}$ designates the outputs of the parties with unmatched passwords as the abort messages without instructions from the simulator.

4 Security of SPAKE2+

In this section, we present the UC security of SPAKE2+ using $\mathcal{F}_{\text{le-aPAKE}}$. Figure 7 shows the SPAKE2+ protocol proposed in [16]. The usage of the session identifier and the user account identifier is modified to fit the discussion of this paper.

4.1 Attack Scenarios

To illustrate why SPAKE2+ does not UC-realize the original aPAKE functionality and clarify the meaning of the relaxations in $\mathcal{F}_{\text{le-aPAKE}}$, we describe two attack scenarios for SPAKE2+. Similar illustrations for the first part were already given by Shoup [36] and Abdalla et al. [1,2].

Password guess after the session is completed. Even after a session is completed, an adversary $\mathcal{A}$ actively attacking SPAKE2+ can verify the correctness of its password guess of a party by impersonating a counterpart of the party. For the sake of simplicity, suppose that $\mathcal{A}$ guesses that the password of the server is pw' . Let (π'_0, π'_1) be the hash value of pw' . If $\mathcal{A}$ chooses $x' \in \mathbb{Z}_p$ and sends $X' = g^{x'} U^{\pi'_0}$ to the server, $\mathcal{A}$ receives Y from the server and can send a query $(sid, \pi'_0, X', Y, (Y/V^{\pi'_0})^{x'}, (Y/V^{\pi'_0})^{\pi'_1})$ to the random oracle H_2 . The response to the query must coincide with the session key of the server if the guess is correct. $\mathcal{A}$ can send the query to the random oracle at any time after $\mathcal{A}$ sees X , so $\mathcal{A}$ can send it even after the session is completed. $\mathcal{A}$ can test the correctness of the guess on the password of the client in the same way. However, in the ideal world, the simulator $\mathcal{S}$ cannot clarify what password is used in X' until $\mathcal{A}$ sends the query to the random oracle H_2 , and the original aPAKE functionality allows $\mathcal{S}$ to make a password guess only before the session is completed. So, $\mathcal{S}$ cannot correctly simulate the random oracle H_2 if $\mathcal{A}$ sends the query to random oracle H_2 after the session is completed.

Hence, in defining a functionality that SPAKE2+ can achieve, the functionality must be relaxed to allow $\mathcal{S}$ in a session to make a password guess after the session is completed. This relaxation is modeled as the **RegisterTest** query and the **LateTestPwd** query in $\mathcal{F}_{\text{le-aPAKE}}$.

Two ways for active attacks: LateTestPwd and Impersonate. An adversary $\mathcal{A}$ that steals the password file does not reveal whether it impersonates the server using the password file or tries to verify a password guess irrelevant to the password file before $\mathcal{A}$ makes a password guess by querying to the random oracle H_2 . Consider the situation that $\mathcal{A}$ in a session steals the password file (π_0, g^{π_1}) and knows $(\pi'_0, \pi'_1) = H_1(uid, pw')$. If $\mathcal{A}$ chooses (π_0, g^{π_1}) or $(\pi'_0, g^{\pi'_1})$ as $(\pi_0^*, g^{\pi_1^*})$ and sends $Y' = g^{y^*} V^{\pi_0^*}$ to the client (y^* is randomly sampled), the simulator $\mathcal{S}$ does not know which password hash is used to calculate Y' . But $\mathcal{A}$ can make a password guess to the client by making a query $(sid, \pi_0^*, X, Y', (X/U^{\pi_0^*})^{y^*}, g^{\pi_1^* y^*})$ to the random oracle H_2 , where X is the message from the client.

Hence, if $\mathcal{A}$ does not make the query before the session is completed, $\mathcal{S}$ has to decide whether to send an **Impersonate** query or a **RegisterTest** query to the ideal functionality without any hints. To address this situation, the **CltSession** message of $\mathcal{F}_{\text{le-aPAKE}}$ contains information about whether the password of a client matches the password of the counterpart server of the client. This information allows $\mathcal{S}$ to decide to use the **Impersonate** query if the password of the client matches the password of the server before the session is completed. Otherwise, $\mathcal{S}$ uses the **RegisterTest** query.

4.2 Proof of Security

Theorem 1. *SPAKE2+ shown in Figure 7 UC-realizes $\mathcal{F}_{\text{le-aPAKE}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{CRS}})$ hybrid model under the GCDH assumption.*

In the same way as Theorem 1, we can prove that SPAKE2+ shown in Figure 7 UC-realizes $\mathcal{F}_{\text{w-le-aPAKE}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{CRS}})$ hybrid model under the GCDH assumption.

In Appendix A, we give the full proof for this theorem. Here, we briefly describe the outline of the proof of Theorem 1.

When the adversary $\mathcal{A}$ in the real world does not steal password files, the security of SPAKE2+ is proven in the same way as the proof of Theorem 1 in [1,2], which states that the PAKE protocol SPAKE2 UC-realizes the lePAKE functionality. The structure of SPAKE2+ can be illustrated as SPAKE2 using π_0 as the password of the parties and additionally using $N = g^{\pi_1 y}$ in calculating the hash value for key generations. Therefore, it can be seen that SPAKE2+ is as secure as SPAKE2 in situations where $\mathcal{A}$ does not steal password files.

In the situation where $\mathcal{A}$ compromises servers and steal their password files, the security of SPAKE2+ is mostly explained in the same way as the security proof of Theorem 3 in [36], which states the aPAKE protocol IETF-SPAKE2+ UC-realizes the variant of r-aPAKE-EA functionality. IETF-SPAKE2+ is depicted as a variant of SPAKE2+ with key confirmation. In the security proof of IETF-SPAKE2+, it is shown that the adversary cannot find the session key of the server in an event named “Post-compromise breaks”. This event captures the situation where the adversary wants to find the session key of a compromised server but the password of the server is not in the record of the queries to the random oracle H_1 .

Intuitively, in this situation, the session key of the server is secure because of the hardness of calculating N from the message of the server Y and the part of the password file L . Consider a reduction that wants to solve the GCDH problem instance (g^{a_1}, g^{a_2}) . The reduction takes $m, n \leftarrow_{\mathbb{R}} \mathbb{Z}_p$ and sets the common reference strings as $U = g^m, V = g^n$. Then, the reduction chooses one client-server pair and sets the problem instance into a session as $Y = g^{a_1}$ and $L = g^{a_2}$. When the adversary $\mathcal{A}$ steals the password file, the reduction sends (π_0, L) to $\mathcal{A}$ (π_0 is randomly chosen by the reduction). During the execution, if $\mathcal{A}$ makes a query to the random oracle H_2 that matches the session key of the server, this query has to contain $N = g^{a_1 a_2} / V^{\pi_0 a_2}$. The reduction checks the correctness of N in the query and, if this check is true, outputs $N \cdot (g^{a_2})^{n \pi_0} = g^{a_1 a_2}$. The DDH oracle in the GCDH assumption is used for this check. In the security analysis of IETF-SPAKE2+, Shoup [36] used the “Trapdoor Test”, which is available under the CDH assumption, so that the reduction can detect when the adversary makes the query that matches the session key of the server.

We note that reductions for this situation cannot be very tight. If the adversary makes a query of the password of the server to the random oracle H_1 , this reduction cannot respond to this query because the reduction does not know $\pi_1 = a_2$. Hence, this reduction cannot use one CDH problem instance for

multiple client-session pairs. It is possible to use one CDH problem instance in multiple sessions between one client-server pair by taking $\tau \leftarrow_{\mathcal{R}} \mathbb{Z}_p$ and setting $Y = g^\tau g^{a_1}$ for each session as did in the security analysis of IETF-SPAKE2+. Shoup [36] indicated that the reductions for the variants of SPAKE2+ cannot be very tight for a similar reason. As a general result, Liu et al. [31,32] showed that aPAKE protocols cannot become tightly secure for a similar reason.

A special case we consider is the situation where the passwords of a client and a server match and the adversary $\mathcal{A}$ sends a message to the client without knowing the hash of the password of the client and steals the password file after the session is completed. Suppose that $\mathcal{A}$ attacking on a session sid receives X from the client and sends Y' to the client before $\mathcal{A}$ steals the password file and queries the password of the client to the random oracle H_1 . Then, if $\mathcal{A}$ steals the password file (π_0, g^{π_1}) , it is hard for $\mathcal{A}$ to make a query $H_2(sid, \pi_0, X, Y', (Y'/V^{\pi_0})^x, (Y'/V^{\pi_0})^{\pi_1})$, which equals the session key of the client.

This hardness can be reduced to the GPCCDH assumption. Consider a reduction that wants to solve the GPCCDH problem instance (A_1, B_1, B_2) . The reduction sets the common reference strings $U = B_1$, and $V = B_2$, chooses one session identifier sid randomly, and sends A_1 to $\mathcal{A}$ as a message sent by the client in the session. If $\mathcal{A}$ sends a message Y' to the client, then the reduction sends a `RegisterTest` query to $\mathcal{F}_{w\text{-le-aPAKE}}$, outputs Y' to the challenger of the GPCCDH problem, and receives $\pi_0^* \in \mathcal{P} = \mathbb{Z}_p$ from the challenger. If $\mathcal{A}$ later makes a `StealPwdFile` query to the server, the reduction takes $\pi_1 \leftarrow_{\mathcal{R}} \mathbb{Z}_p$ and returns (π_0^*, g^{π_1}) . If $\mathcal{A}$ makes a query $H_2(sid, \pi_0^*, A_1, Y', Z', N')$ to the random oracle, then the reduction checks $\text{DDH}(A_1/B_1^{\pi_0^*}, Y'/B_2^{\pi_0^*}, Z') = 1$ using the DDH oracle and $N' = (Y'/B_2^{\pi_0^*})^{\pi_1}$. If this check is true, then the reduction outputs Z' .

Regarding this special case, there are four points we note.

- This reduction is not tight because this reduction must choose one session randomly and send Y' to the challenger before $\mathcal{A}$ makes the query to the random oracle H_2 .
- This reduction does not utilize $N' = (Y'/B_2^{\pi_0^*})^{\pi_1}$ in the query to the random oracle H_2 for calculating Z' . $\mathcal{A}$ can naturally calculate N' when $\mathcal{A}$ knows π_1 from the random oracle H_1 .
- The security of SPAKE2+ in this special case is achieved because SPAKE2+ has a common structure with SPAKE2. The GPCCDH assumption was used to show perfect forward secrecy of SPAKE2 in [1,2], and this reduction applies this assumption to the common structure in SPAKE2+.
- When we try to prove that SPAKE2+ realizes $\mathcal{F}_{w\text{-le-aPAKE}}$, which states that SPAKE2+ UC-realizes $\mathcal{F}_{w\text{-le-aPAKE}}$, this special case is ignored. This is because, in this special case, the simulator interacting with $\mathcal{F}_{w\text{-le-aPAKE}}$ can receive the session key of the client by sending a `LateTestPwd` query with $\perp$ to $\mathcal{F}_{w\text{-le-aPAKE}}$ whether $\mathcal{A}$ makes the query to the random oracle H_2 or not.

Assumption. We use the GCDH assumption to prove the UC security of SPAKE2+ as well as the security proof of SPAKE2 by Abdalla et al. [1,2].

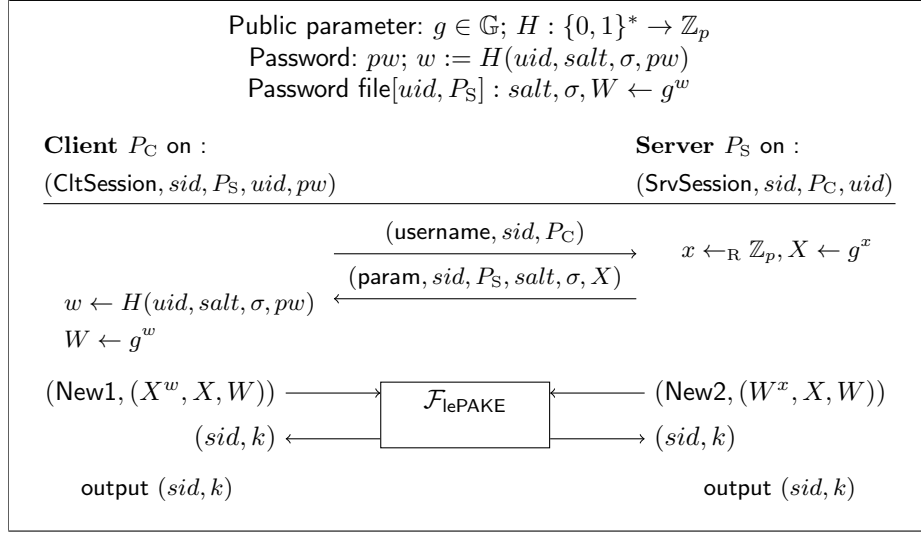


Fig. 8. The le-AuCPace protocol. $(\text{New1}, (X^w, X, W))$ is an abbreviation of $(\text{NewSession}, sid, P_C, P_S, (X^w, X, W), \text{clt})$. $(\text{New2}, (W^x, X, W))$ is an abbreviation of $(\text{NewSession}, sid, P_S, P_C, (W^x, X, W), \text{srv})$. The original definition of le-AuCPace is in [21,22]. We make some changes to the original definition as mentioned in Section 5.

However, the strong Diffie-Hellman assumption, which is a weaker assumption than the GCDH assumption, is used in the security proofs of KC-SPAKE2, KC-SPAKE2+, and IETF-SPAKE2+ by Shoup [36]. We determine to use the GCDH assumption in this proof to simply introduce the GPCCDH assumption.

When we prove that SPAKE2+ realizes $\mathcal{F}_{\text{w-le-aPAKE}}$. Based on the explanation of the special case in Theorem 1, when we prove that SPAKE2+ realizes $\mathcal{F}_{\text{w-le-aPAKE}}$, we do not use the non-tight reduction to the GPCCDH assumption. In addition, the simulator in this proof does not exploit the D value in the CltSession message. This is because, in a session where the adversary steals the password file, the simulator can send a LateTestPwd query with $\perp$ if the adversary makes a random oracle query that matches the client's session key after the client outputs the session key.

5 Security of le-AuCPace

The le-AuCPace protocol uses the lePAKE protocol as a component. The definition of $\mathcal{F}_{\text{lePAKE}}$ in this paper is the same as the one defined by Abdalla et al. [1,2]. We show the description of $\mathcal{F}_{\text{lePAKE}}$ in Section 2.2.

The le-AuCPace protocol is depicted in Figure 8. In this definition, we make some changes to the original definition of le-AuCPace presented by Haase and

Labrique [21,22]. The usage of the session identifier and the user account identifier is adjusted to suit the discussion of this paper. Also, we consider a cyclic group $\mathbb{G}$ of prime order p and a generator g of $\mathbb{G}$ in this protocol for simplicity.

A major change is that, in the original definition, parties only include $g^{xw} = X^w = W^x$ in `NewSession` queries to $\mathcal{F}_{\text{lePAKE}}$, but in the definition of this paper, they additionally include X and W in the queries. This change is beneficial to ensure that the session keys of the client and the server are different if the passwords at the client and the server do not match or if the values in the `param` message are changed by the adversary. For example, consider the situation where the server stores $(uid, salt, \sigma, W = g^{H(uid, salt, \sigma, pw)})$ and the client uses password pw' ($pw \neq pw'$) in the original definition. In the execution of `leAuCPace`, when the server sends $(\text{param}, sid, P, salt, \sigma, X)$, the adversary can calculate $w = H(uid, salt, \sigma, pw)$ and $w' = H(uid, salt, \sigma, pw')$ and send $(\text{param}, sid, P, salt, \sigma, X^{w/w'})$ to the client. When the adversary behaves in this way, the server and the client use X^w in the execution of `lePAKE` and, at the end of this session, the server and the client have the same session key. However, in the ideal world, the simulator has to identify w and w' from $X^{w/w'}$ to designate the session keys of the parties. To avoid constructing such a simulator, X and W are included in the `NewSession` queries in our definition.

5.1 Why `leAuCPace` Does Not Realize $\mathcal{F}_{\text{le-aPAKE}}$

The reason why `leAuCPace` does not realize $\mathcal{F}_{\text{le-aPAKE}}$ is that, if an adversary $\mathcal{A}$ actively attacking `leAuCPace` steals the password file in a session even after the session is completed, $\mathcal{A}$ can successfully impersonate the server. We consider the situation where $\mathcal{A}$ performs a man-in-the-middle attack on a session where the client and the server use a shared password. When $\mathcal{A}$ receives a message $(\text{param}, sid, P, salt, \sigma, X)$ sent by the server, $\mathcal{A}$ can take $x' \leftarrow \mathbb{Z}_p$ and send $(\text{param}, sid, P, salt, \sigma, X' = g^{x'})$ to the client and a `RegisterTest` query to $\mathcal{F}_{\text{lePAKE}}$. After the session is completed, $\mathcal{A}$ can steal the password file $(salt, \sigma, W)$ and calculate $(W^{x'}, X', W)$, which corresponds to the password of the client in $\mathcal{F}_{\text{lePAKE}}$. Using this result of calculation, $\mathcal{A}$ can send $(\text{LateTestPwd}, sid, P_C, (W^{x'}, X', W))$ to $\mathcal{F}_{\text{lePAKE}}$ and receive the session key of the client.

However, in the ideal world, if the simulator $\mathcal{S}$ interacts with $\mathcal{F}_{\text{le-aPAKE}}$, $\mathcal{S}$ cannot simulate this case because $\mathcal{S}$ cannot extract the password of the client and the server from the message sent by $\mathcal{A}$. To simulate this case, it is needed to allow $\mathcal{S}$ to find the session key of the client when the password file is stolen after the session is completed and the client uses the password matching the password of the server. To allow this, the `LateTestPwd` query of $\mathcal{F}_{\text{w-le-aPAKE}}$ is modeled.

5.2 Proof of Security

Theorem 2. *leAuCPace shown in Figure 8 UC-realizes $\mathcal{F}_{\text{w-le-aPAKE}}$ in the $(\mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{lePAKE}})$ hybrid model under the GCDH assumption.*

We give the full proof in Appendix B. We give the more formal definition of the simulator in Figures 14, 15, and 16, but here we give the idea of the construction of the simulator and the main points of the proof briefly. Because the common protocol is targeted, our proof has some similarities with [20,21,22]. However, this is the first proof using $\mathcal{F}_{w\text{-le-aPAKE}}$.

$\mathcal{S}$ simulates the behavior of clients and servers, remembering $(salt, \sigma, X)$ used by each party and the password file of the servers. For a client, $\mathcal{S}$ sends a `username` message to the adversary $\mathcal{A}$ when a `CltSession` message of the client is sent by $\mathcal{F}_{w\text{-le-aPAKE}}$. Then, if $\mathcal{A}$ sends a `param` message to the client, $\mathcal{S}$ stores $(salt, \sigma, X)$ in the message and sends a `NewSession` message to $\mathcal{A}$ as a message sent by $\mathcal{F}_{\text{lePAKE}}$. For a server, $\mathcal{S}$ generates $salt$ randomly and stores $(\text{FILE}, salt, \sigma, *)$ at the time a `StealPwdFile` message or a `SrvSession` message of the server is first sent by $\mathcal{F}_{w\text{-le-aPAKE}}$. At a later time, If $\mathcal{S}$ receives a `username` message sent by $\mathcal{A}$ to the server, $\mathcal{S}$ takes $x \leftarrow_{\mathbb{R}} \mathbb{Z}_p$, sends a `param` message including $(salt, \sigma, X = g^x)$, stores $(salt, \sigma, X)$, and sends a `NewSession` message to $\mathcal{A}$ as a message sent by $\mathcal{F}_{\text{lePAKE}}$.

When a `StealPwdFile` query is sent to a server by $\mathcal{A}$, the simulator $\mathcal{S}$ passes this query to $\mathcal{F}_{w\text{-le-aPAKE}}$. If the response from the functionality is pw , then $\mathcal{S}$ retrieves the password file $(\text{FILE}, salt, \sigma, *)$, searches the hash value $w = H(uid, salt, \sigma, pw)$, and sends $(uid, salt, \sigma, W = g^w)$ to $\mathcal{A}$. $\mathcal{S}$ updates the password file to $(\text{FILE}, salt, \sigma, W)$. Otherwise, $\mathcal{S}$ takes $w^* \leftarrow_{\mathbb{R}} \mathbb{Z}_p$, sends $(uid, salt, \sigma, W^* = g^{w^*})$ to $\mathcal{A}$, and stores $(\text{FILE}, salt, \sigma, W^*)$ as the password file of the server.

When the adversary $\mathcal{A}$ queries $(uid, salt, \sigma, pw)$ to the random oracle H , if the hash value of this query is already generated, $\mathcal{S}$ sends it to $\mathcal{A}$. If not, the simulator $\mathcal{S}$ checks whether the server of uid is already initiated by the `StorePwdFile` message and the password file of this server is $(\text{FILE}, salt, \sigma, *)$. If the result of this check is true, $\mathcal{S}$ sends $(\text{OfflineTestPwd}, P_s, uid, pw)$ to $\mathcal{F}_{w\text{-le-aPAKE}}$. If the response from $\mathcal{F}_{w\text{-le-aPAKE}}$ is “correct guess”, $\mathcal{S}$ retrieves the password file $(\text{FILE}, salt, \sigma, W = g^w)$ and returns w to $\mathcal{A}$. If $\mathcal{S}$ does not send a response to $\mathcal{A}$ in the above procedure, $\mathcal{S}$ samples $h \leftarrow_{\mathbb{R}} \mathbb{Z}_p$, returns h to $\mathcal{A}$, and stores h as the hash value of this query.

When the adversary $\mathcal{A}$ sends a `RegisterTest` query, the simulator $\mathcal{S}$ passes this message to $\mathcal{F}_{w\text{-le-aPAKE}}$.

When the adversary $\mathcal{A}$ makes a password guess by sending $(\text{TestPwd}, sid, P, (Z', X', W'))$ or $(\text{LateTestPwd}, sid, P, (Z', X', W'))$ to $\mathcal{F}_{\text{lePAKE}}$, where X' equals X used in the party P , the simulator $\mathcal{S}$ works as follows.

- $\mathcal{S}$ retrieves uid , the other party P' , and $(salt, \sigma, X)$ used in the attacked party P in this session. $\mathcal{S}$ also searches pw such that $Z' = X'^{H(uid, salt, \sigma, pw)}$ and $W' = g^{H(uid, salt, \sigma, pw)}$ in the record of the random oracle H .
 - If pw is found, $\mathcal{S}$ sends $(\text{TestPwd}, sid, P, pw)$ or $(\text{LateTestPwd}, sid, P, P', uid, pw)$ to $\mathcal{F}_{w\text{-le-aPAKE}}$, and passes the response from $\mathcal{F}_{w\text{-le-aPAKE}}$ to $\mathcal{A}$. $\mathcal{S}$ uses the same type of query that $\mathcal{A}$ uses.
 - If not, $\mathcal{S}$ checks whether the password file $(\text{FILE}, salt, \sigma, W = g^w)$ is used to calculate Z' by checking whether the role of P is the client, the password file of the session is stolen, and (Z', W') equals (X'^w, W) .

- * If the result of check is true, $\mathcal{S}$ sends $(\text{Impersonate}, \text{sid}, P, P', \text{uid})$ or $(\text{LateTestPwd}, \text{sid}, P, P', \text{uid}, \perp)$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ and passes the response from $\mathcal{F}_{\text{w-le-aPAKE}}$ to $\mathcal{A}$. If $\mathcal{A}$ uses the TestPwd query, $\mathcal{S}$ uses the Impersonate query. Otherwise, $\mathcal{S}$ uses the LateTestPwd query.
- * Otherwise, $\mathcal{S}$ sends $(\text{TestPwd}, \text{sid}, P, 0)$ or $(\text{LateTestPwd}, \text{sid}, P, P', \text{uid}, 0)$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ and passes the response sent by $\mathcal{F}_{\text{w-le-aPAKE}}$ to $\mathcal{A}$. $\mathcal{S}$ uses the same type of query that $\mathcal{A}$ uses.

When $\mathcal{S}$ receives $(\text{NewKey}, \text{sid}, P, k)$ from $\mathcal{A}$, $\mathcal{S}$ checks whether the values used as (salt, σ, X) in the client and the server in this session do not match, and the state of $\langle \text{sid}, P, *, *, \text{clt} \rangle$ in $\mathcal{F}_{\text{w-le-aPAKE}}$ is **fresh**. If the check is true, $\mathcal{S}$ sends $(\text{TestPwd}, \text{sid}, P, \perp)$ to $\mathcal{F}_{\text{w-le-aPAKE}}$. $\mathcal{S}$ then simply passes the NewKey query from $\mathcal{A}$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ and the response from $\mathcal{F}_{\text{w-le-aPAKE}}$ to $\mathcal{A}$.

The GCDH assumption is used to prove that when the adversary $\mathcal{A}$ steals the password file $W = g^w$ and sees X in the **param** message sent by the server, it is hard for $\mathcal{A}$ to calculate X^w , which is used in the password of the server in $\mathcal{F}_{\text{lePAKE}}$, without knowing w . To prove the hardness of calculating W^x , consider a reduction that wants to calculate $g^{a_1 a_2}$ from (g^{a_1}, g^{a_2}) . The reduction chooses one client-server pair i.e. one uid and sets $W = g^{a_1}$ as the password file of the server. When $\mathcal{A}$ sends $(\text{username}, \text{sid}, P)$ as a message sent by the client, the reduction samples $\beta_{\text{sid}} \leftarrow_{\mathbb{R}} \mathbb{Z}_p$ and sends $(\text{param}, \text{sid}, P', \text{salt}, \sigma, X = g^{a_2} g^{\beta_{\text{sid}}})$ to $\mathcal{A}$. If $\mathcal{A}$ steals the password file W and sends $(\text{TestPwd}, \text{sid}, P, (Z, X, W))$ or $(\text{LateTestPwd}, \text{sid}, P, (Z, X, W))$ to $\mathcal{F}_{\text{lePAKE}}$ without querying the correct password to the random oracle H , the reduction checks whether $\text{DDH}(g^{a_1}, g^{a_2}, Z/g^{a_1 \beta_{\text{sid}}}) = 1$ or 0 using the DDH oracle. If the answer is 1, the reduction outputs $Z/g^{a_1 \beta_{\text{sid}}}$. Otherwise, the reduction sends $(\text{TestPwd}, \text{sid}, P, 0)$ or $(\text{LateTestPwd}, \text{sid}, P, P', \text{uid}, 0)$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ and passes the response from the functionality to $\mathcal{A}$. The reduction handles other sessions between the same client-server pair in the same way, changing β_{sid} for each session.

We note that this reduction is not tight because of the limitation of aPAKE protocols proved by Liu et al. [31,32]. If the adversary makes a query to the random oracle $H(\text{uid}, \text{salt}, \sigma, pw)$, where pw is the password of the server and salt, σ is the parameter of the server, the reduction must return a_1 , but the reduction does not know a_1 .

6 Compiler from le-aPAKE or w-le-aPAKE to r-aPAKE-EA

In this section, we discuss the compiler from le-aPAKE or w-le-aPAKE to r-aPAKE-EA. Figure 9 shows the r-aPAKE-EA protocol Π , which uses the functionality of le-aPAKE or w-le-aPAKE as a component. The idea for constructing this protocol is the same as the idea of the compiler from lePAKE to rPAKE-EA presented by Abdalla et al. [1,2].

Theorem 3. *The protocol Π shown in Figure 9 UC-realizes $\mathcal{F}_{\text{r-aPAKE-EA}}$ in the $\mathcal{F}_{\text{le-aPAKE}}$ or $\mathcal{F}_{\text{w-le-aPAKE}}$ hybrid model if PRF is a secure PRF.*

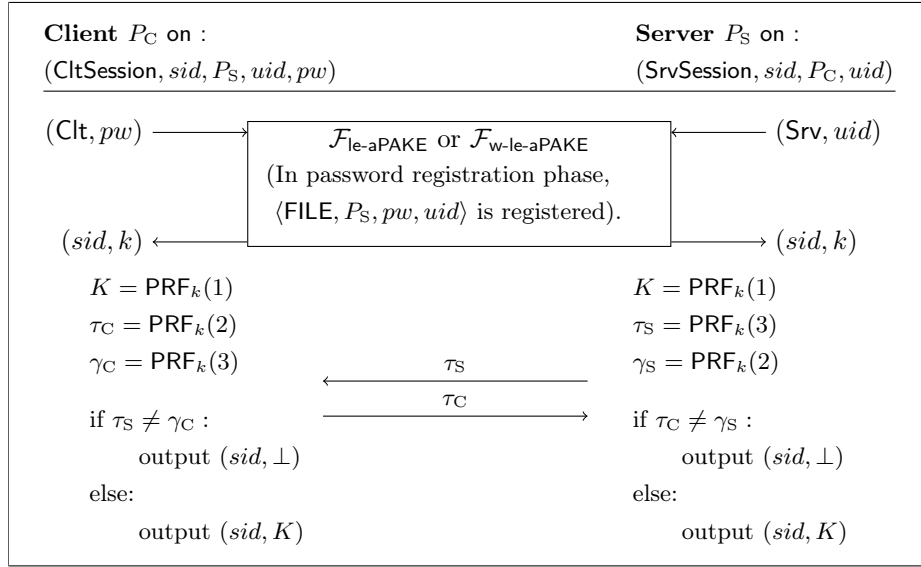


Fig. 9. The compiler from the le-aPAKE protocol to the r-aPAKE-EA protocol Π . (Clt, pw) is an abbreviation of (CltSession, sid, P_S, uid, pw). (Srv, uid) is an abbreviation of (SrvSession, sid, P_C, uid). This construction is based on a compiler from lePAKE to rPAKE defined in [1,2].

We describe the full proof in Appendix C. Here, we give a brief explanation of the proof of this theorem. Since $\mathcal{F}_{w-le-aPAKE}$ is weaker than $\mathcal{F}_{le-aPAKE}$, it is sufficient to show that Π UC-realizes $\mathcal{F}_{r-aPAKE-EA}$ in the $\mathcal{F}_{w-le-aPAKE}$ hybrid model to prove this theorem.

The proof of this theorem is almost the same as the proof of Theorem 2 in the study by Abdalla et al. [1,2], which states that lePAKE with key confirmation UC-realizes the functionality of rPAKE-EA, and the proof covers the cases where the adversary does not steal password files in Theorem 3 of this section.

If the adversary attacking Π makes a StealPwdFile query or an OfflineTestPwd query, the simulator passes this query to $\mathcal{F}_{w-le-aPAKE}$ and the response from $\mathcal{F}_{w-le-aPAKE}$ to the adversary.

To roughly show the construction of simulator $\mathcal{S}$, we discuss two situations where the adversary $\mathcal{A}$ steals password files. To consider the situations where $\mathcal{A}$ utilizes stolen password files, we assume that the value D in the CltSession message of the attacked session is 1.

When the adversary uses Impersonate queries. In the real world, if $\mathcal{A}$ steals the password file and sends an Impersonate query to $\mathcal{F}_{w-le-aPAKE}$ before the session of $\mathcal{F}_{w-le-aPAKE}$ is completed, $\mathcal{A}$ can designate the session key k of the client in $\mathcal{F}_{w-le-aPAKE}$ by sending NewKey query to $\mathcal{F}_{w-le-aPAKE}$. Using this session key k , $\mathcal{A}$ can calculate $PRF_k(3) = \gamma_C = \tau_S$ and send this value to the client as the key

confirmation message from the server. If $\mathcal{A}$ does this behavior, the client finally outputs $\text{PRF}_k(1)$ as the session key. Otherwise, the client finally outputs $\perp$. Also, $\mathcal{A}$ can validate the key confirmation message τ_C sent by the client by calculating $\text{PRF}_k(2)$ and comparing it with τ_C .

In the ideal world, when $\mathcal{A}$ sends the **Impersonate** query, the simulator $\mathcal{S}$ sends the **Impersonate** query to $\mathcal{F}_{\text{r-aPAKE-EA}}$ and passes the response from $\mathcal{F}_{\text{r-aPAKE-EA}}$ to $\mathcal{A}$. Then, when $\mathcal{S}$ receives the **NewKey** query with the session key k made by $\mathcal{A}$, $\mathcal{S}$ sends the $\text{PRF}_k(2)$ as a key confirmation message τ_C from the client to $\mathcal{A}$. If $\mathcal{A}$ sends a key confirmation message to the client, $\mathcal{S}$ checks whether the value of the message equals $\text{PRF}_k(3)$. If this check is true, $\mathcal{S}$ sends a **NewKey** query with $\text{PRF}_k(1)$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$. Otherwise, $\mathcal{S}$ sends a **NewKey** query with $\perp$.

When the adversary uses RegisterTest queries. Consider the situation where $\mathcal{A}$ attacks a session in the real world and wants to find the session key of the client. $\mathcal{A}$ can send a **RegisterTest** query to $\mathcal{F}_{\text{w-le-aPAKE}}$ before the session of $\mathcal{F}_{\text{w-le-aPAKE}}$ is completed. After $\mathcal{A}$ sends a **NewKey** query, $\mathcal{A}$ can steal the password file, send a **LateTestPwd** query with $\perp$ to $\mathcal{F}_{\text{w-le-aPAKE}}$, and receive the session key of the client in $\mathcal{F}_{\text{w-le-aPAKE}}$ as the response.

In the ideal world, if $\mathcal{A}$ makes the **NewKey** query, $\mathcal{S}$ chooses k' randomly and records k' as the session key of the client outputted by $\mathcal{F}_{\text{w-le-aPAKE}}$. $\mathcal{S}$ sends $\text{PRF}_{k'}(2)$ to $\mathcal{A}$ as the key confirmation message τ_C sent by the client. If $\mathcal{A}$ sends the **LateTestPwd** query with $\perp$, $\mathcal{S}$ checks whether the password file is stolen and D in the **CltSession** message is 1. If this check is true, $\mathcal{S}$ sends k' to $\mathcal{A}$. Otherwise, $\mathcal{S}$ picks $k^* \leftarrow_{\text{R}} \{0, 1\}^\kappa$ and sends k^* . When $\mathcal{A}$ later sends a key confirmation message to the client, $\mathcal{S}$ checks whether the message equals $\text{PRF}_{k'}(3)$. If the result of this check is true, $\mathcal{S}$ sends an **Impersonate** query and a **NewKey** query with $\text{PRF}_{k'}(1)$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$. Otherwise, $\mathcal{S}$ sends a **NewKey** query with $\perp$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$. If $\mathcal{A}$ sends a key confirmation message to the client before $\mathcal{A}$ sends the **LateTestPwd** query, $\mathcal{A}$ does not know k' and cannot find $\text{PRF}_{k'}(3)$. So, $\mathcal{S}$ sends a **NewKey** query with $\perp$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$.

7 Variations of r-aPAKE-EA

In this section, we discuss the two variants of r-aPAKE-EA which are satisfiable by w-le-aPAKE with key confirmation.

Here, we give two preliminaries for this section. First, we name the variants of the **CltSession** interface described in Section 3. We call the **CltSession** interface which tells the adversary the information on whether a client and a server use a common password as a relaxed **CltSession** interface. The relaxed **CltSession** interface is introduced in the functionalities defined in Section 3. We call the **CltSession** interface which does not tell the information as a non-relaxed **CltSession** interface. The non-relaxed **CltSession** interface is used in the original aPAKE functionality proposed by Gentry et al. [17].

Second, we assume that the w-le-aPAKE functionality in this section has the non-relaxed **CltSession** interface. As explained in Section 3, we consider whether

or not a w-le-aPAKE has a relaxed `CltSession` interface is not relevant to the capability of the simulation in the ideal world in the proofs we consider in this paper. Theorem 1, which states that SPAKE2+ UC-realizes $\mathcal{F}_{w\text{-le-aPAKE}}$, holds when $\mathcal{F}_{w\text{-le-aPAKE}}$ with non-relaxed `CltSession` is used as described in Section 4.2. Theorem 2, which states that le-AuCPace UC-realizes $\mathcal{F}_{w\text{-le-aPAKE}}$, also holds if w-le-aPAKE has the non-relaxed `CltSession` interface as can be seen from the proof in Section 5.2.

7.1 Proposed Definitions

In this section, we propose the two variants of r-aPAKE-EA which are satisfied by $\mathcal{F}_{w\text{-le-aPAKE}}$ with the non-relaxed `CltSession` interface accompanied by the key confirmation process.

The difference between the two variants and r-aPAKE is how the functionality sends the simulator the information needed to simulate the adversary sending `LateTestPwd` with $\perp$. When the adversary steals a password file and the client and the server use the same password, the adversary can gain the session key of the client by sending a `LateTestPwd` query with $\perp$ to $\mathcal{F}_{w\text{-le-aPAKE}}$. Since the simulator knows whether the password file has been stolen, the simulator can determine whether the adversary obtains the session key if it knows whether the client and the server use a common password.

We propose three methods to define functionalities achievable by w-le-aPAKE with key confirmation.

Method 1: The functionality always provides whether the passwords of a client and a server match or not.

This method uses r-aPAKE-EA, which is defined in Section 3. r-aPAKE-EA has the relaxed `CltSession` interface. So, when a `CltSession` query is sent to the client, the simulator knows whether the client and the server use a common password or not.

On $(\text{LateTestPwd}, sid, P, P', uid, pw^*)$ from $\mathcal{S}$, if there is a record $\langle sid, P, P', pw, role, K \rangle$ marked completed and flagged tested, remove the flag from the record and do:

- If $pw^* = pw$, then return “correct guess” to $\mathcal{S}$.
- If $pw^* = \perp$, $role = \text{clt}$, $\langle \text{FILE}, P', uid, pw' \rangle$ is marked compromised, and $pw = pw'$, then return “correct guess” to $\mathcal{S}$.
- Otherwise, return “wrong guess” to $\mathcal{S}$.

Fig. 10. The `LateTestPwd` interface used in Method 2. The shadowed text shows the difference from the `LateTestPwd` interface in r-aPAKE-EA defined in Section 3.

Method 2: The functionality conditionally provides whether the passwords of a client and a server match or not.

In this method, the `LateTestPwd` interface defined in Figure 10 is incorporated into $\mathcal{F}_{\text{r-aPAKE-EA}}$ with the non-relaxed `CltSession` interface. When a `LateTestPwd` query with $\perp$ is sent to the client, if the client and the server use the same password and the password file is stolen, the functionality sends “correct guess” to the simulator. This functionality can be interpreted as a variant of the $\mathcal{F}_{\text{w-le-aPAKE}}$ with the non-relaxed `CltSession` interface whose `LateTestPwd` interface is transformed in the way of relaxation in r-aPAKE.

Method 3: The functionality does not provide whether passwords match or not, but uses restricted key confirmation.

This method uses $\mathcal{F}_{\text{r-aPAKE-EA}}$ with the non-relaxed `CltSession` and imposes a restriction on the key confirmation process that is attached to $\mathcal{F}_{\text{w-le-aPAKE}}$. In the restricted key confirmation, clients send their key confirmation messages after receiving the key confirmation messages from servers. Concretely, the procedure of the restricted key confirmation is as follows.

1. The server and the client receive a key K from $\mathcal{F}_{\text{w-le-aPAKE}}$.
2. The server sends $\tau_S = \text{PRF}_K(3)$ to the client.
3. The client checks whether $\tau_S = \text{PRF}_K(3)$. If this check passes, the client sends $\tau_C = \text{PRF}_K(2)$ and output $K' = \text{PRF}_K(1)$ as the session key. Otherwise, the client outputs `abort` and does not send any message to the server.
4. The server check whether $\tau_C = \text{PRF}_K(2)$. If this check passes, the server output $K' = \text{PRF}_K(1)$ as the session key. Otherwise, the server outputs `abort`.

$\mathcal{F}_{\text{w-le-aPAKE}}$ with this restricted key confirmation realizes $\mathcal{F}_{\text{r-aPAKE-EA}}$ with the non-relaxed `CltSession` interface. This is because the adversary impersonating the server must send the key confirmation message before the client outputs the key confirmation message and, to send correctly, the adversary must know K by querying `TestPwd`, `LateTestPwd`, or `Impersonate`.

7.2 Discussion

Compared to Method 1 and Method 2, Method 3 is interpreted as a method that modifies the structure of the aPAKE functionality as little as possible from the original aPAKE functionality defined by Gentry et al. [17]. Method 1 has the weakest security and Method 3 has the strongest security in the proposed three methods.

We discuss some features of the three methods in the following three sections.

Validity of Security Relaxation in Method 1. Here, we consider the validity of the security of Method 1, which is the weakest secure. In this method, the aPAKE functionality introduces the relaxed `CltSession` interface. In [36] Remark 3, Shoup argued that there is no problem in incorporating the relaxed

CltSession interface in the functionalities of PAKE and aPAKE. This argument is also applicable to $\mathcal{F}_{\text{r-aPAKE-EA}}$ we defined. Here, we illustrate the argument by Shoup as a discussion of r-aPAKE-EA.

Consider a case where a client and a server do the key exchange using $\mathcal{F}_{\text{r-aPAKE-EA}}$ and later they send ciphertexts encrypting using the session key generated by the key exchange. The client and the server are assumed to generate the session key while maintaining their **fresh** state in $\mathcal{F}_{\text{r-aPAKE-EA}}$. If the passwords match, the client and the server obtain the session key from the $\mathcal{F}_{\text{r-aPAKE-EA}}$. Otherwise, the key exchange fails, and the client and the server output **abort**. Next to this key exchange, if the session key is generated, the parties send the ciphertexts encrypted with the session key, or else they do not send the ciphertexts.

To study the security of the key exchange and sending of the ciphertexts as described above, we consider constructing an ideal functionality $\mathcal{F}$ that represents secure communication with r-aPAKE-EA. The password-based secure channel functionality defined by Canetti et al. [13] is defined for the same purpose as $\mathcal{F}$, and it is illustrated as an example in the description by Shoup [36]. Assume that $\mathcal{F}$ receives the passwords of the parties and the plaintexts from the environment. To prove that the communication with aPAKE is secure, we consider constructing a simulator that interacts with $\mathcal{F}$. The simulator simulates the key exchange with $\mathcal{F}_{\text{r-aPAKE-EA}}$ and the sending of the ciphertexts. To simulate, the simulator must decide whether to send the ciphertexts. Therefore, $\mathcal{F}$ needs to inform the simulator whether the passwords of the parties match or not, which shows whether the key exchanges are successful or not in **fresh** sessions.

Considering the information that needs to be given to $\mathcal{F}$, which can be seen from the above discussion, whether $\mathcal{F}_{\text{r-aPAKE-EA}}$ gives the simulator the information on whether the passwords of the server and the client match does not affect whether $\mathcal{F}$ can be realized. Even if $\mathcal{F}_{\text{r-aPAKE-EA}}$ does not provide that information to the simulator, $\mathcal{F}$ must give the simulator whether the client and the server use a common password. Thus, $\mathcal{F}$ is not affected by whether or not the structure of $\mathcal{F}_{\text{r-aPAKE-EA}}$ has the relaxed CltSession interface or the non-relaxed CltSession interface.

Therefore, when considering $\mathcal{F}_{\text{r-aPAKE-EA}}$ as a component to realize a functionality whose behavior is affected by whether the password authentication is successful or not, it is considered that using the relaxed CltSession interface as Method 1 has no effect on the functionality that can be realized.

Relationship between Method 1 and Method 2. Compared to Method 1, Method 2 gives less information to the simulator. Since both methods are satisfied by $\mathcal{F}_{\text{w-le-aPAKE}}$ with the key confirmation, Method 2 captures some features of the security of $\mathcal{F}_{\text{w-le-aPAKE}}$ with the key confirmation that are not captured in Method 1. Specifically, for the functionality in Method 2, unless the simulator steals a password file and sends an **Impersonate** query or a **LateTestPwd** query with $\perp$, the simulator cannot know if the client and the server use a common password. Considering the capability of the simulator, an adversary attacking the functionality in Method 2 does not know whether the passwords of the server

and the client match for sessions where the adversary performs password guesses, such as sending `TestPwd` queries or `LateTestPwd` queries with passwords.

Number of Rounds of Restriction for Key Confirmation in Method 3.

The disadvantage of using Method 3 is the number of rounds in the overall aPAKE protocol with the restricted key confirmation could increase. The non-restricted key confirmation, where the server and the client send their key confirmation messages to the other asynchronously, is executed in one round. However, the key confirmation process in Method 3 could be executed in two rounds because the client needs to wait for the arrival of the key confirmation message of the server.

However, in the case of an aPAKE protocol where the server sends the last message, the disadvantage is solved by sending the last message and the key confirmation message from the server simultaneously. Such a technique for reducing the number of rounds is seen, for example, in the PAKE protocol SPAKE2 with key confirmation shown in [2] Figure 7 and the variants of SPAKE2 and SPAKE2+ shown in [36].

References

1. M. Abdalla, M. Barbosa, T. Bradley, S. Jarecki, J. Katz, and J. Xu. Universally composable relaxed password authenticated key exchange. In D. Micciancio and T. Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020, Part I*, volume 12170 of *Lecture Notes in Computer Science*, pages 278–307. Springer, Cham, Aug. 2020. doi:10.1007/978-3-030-56784-2_10.
2. M. Abdalla, M. Barbosa, T. Bradley, S. Jarecki, J. Katz, and J. Xu. Universally composable relaxed password authenticated key exchange. Cryptology ePrint Archive, Report 2020/320, 2020. URL: <https://eprint.iacr.org/2020/320>.
3. M. Abdalla, M. Barbosa, J. Katz, J. Loss, and J. Xu. Algebraic adversaries in the universal composability framework. In M. Tibouchi and H. Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part III*, volume 13092 of *Lecture Notes in Computer Science*, pages 311–341. Springer, Cham, Dec. 2021. doi:10.1007/978-3-030-92078-4_11.
4. M. Abdalla, B. Haase, and J. Hesse. Security analysis of CPace. In M. Tibouchi and H. Wang, editors, *Advances in Cryptology – ASIACRYPT 2021, Part IV*, volume 13093 of *Lecture Notes in Computer Science*, pages 711–741. Springer, Cham, Dec. 2021. doi:10.1007/978-3-030-92068-5_24.
5. M. Abdalla and D. Pointcheval. Simple password-based encrypted key exchange protocols. In A. Menezes, editor, *Topics in Cryptology – CT-RSA 2005*, volume 3376 of *Lecture Notes in Computer Science*, pages 191–208. Springer, Berlin, Heidelberg, Feb. 2005. doi:10.1007/978-3-540-30574-3_14.
6. M. Bellare, D. Pointcheval, and P. Rogaway. Authenticated key exchange secure against dictionary attacks. In B. Preneel, editor, *Advances in Cryptology – EUROCRYPT 2000*, volume 1807 of *Lecture Notes in Computer Science*, pages 139–155. Springer, Berlin, Heidelberg, May 2000. doi:10.1007/3-540-45539-6_11.

7. S. M. Bellovin and M. Merritt. Encrypted key exchange: Password-based protocols secure against dictionary attacks. In *1992 IEEE Symposium on Security and Privacy*, pages 72–84. IEEE Computer Society Press, May 1992. doi:10.1109/RISP.1992.213269.
8. S. M. Bellovin and M. Merritt. Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise. In D. E. Denning, R. Pyle, R. Ganesan, R. S. Sandhu, and V. Ashby, editors, *ACM CCS 1993: 1st Conference on Computer and Communications Security*, pages 244–250. ACM Press, Nov. 1993. doi:10.1145/168588.168618.
9. F. Benhamouda and D. Pointcheval. Verifier-based password-authenticated key exchange: New models and constructions. Cryptology ePrint Archive, Report 2013/833, 2013. URL: <https://eprint.iacr.org/2013/833>.
10. R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. Cryptology ePrint Archive, Report 2000/067, 2000. URL: <https://eprint.iacr.org/2000/067>.
11. R. Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *FOCS 2001: 42nd Annual Symposium on Foundations of Computer Science*, pages 136–145. IEEE Computer Society Press, Oct. 2001. doi:10.1109/SFCS.2001.959888.
12. R. Canetti, Y. Dodis, R. Pass, and S. Walfish. Universally composable security with global setup. In S. P. Vadhan, editor, *TCC 2007: 4th Theory of Cryptography Conference*, volume 4392 of *Lecture Notes in Computer Science*, pages 61–85. Springer, Berlin, Heidelberg, Feb. 2007. doi:10.1007/978-3-540-70936-7_4.
13. R. Canetti, S. Halevi, J. Katz, Y. Lindell, and P. D. MacKenzie. Universally composable password-based key exchange. In R. Cramer, editor, *Advances in Cryptology – EUROCRYPT 2005*, volume 3494 of *Lecture Notes in Computer Science*, pages 404–421. Springer, Berlin, Heidelberg, May 2005. doi:10.1007/11426639_24.
14. R. Canetti, Y. Lindell, R. Ostrovsky, and A. Sahai. Universally composable two-party and multi-party secure computation. In J. H. Reif, editor, *STOC 2002: 34th Annual ACM Symposium on Theory of Computing*, pages 494–503. ACM Press, May 2002. doi:10.1145/509907.509980.
15. R. Canetti, Y. Lindell, R. Ostrovsky, and A. Sahai. Universally composable two-party and multi-party secure computation. Cryptology ePrint Archive, Report 2002/140, 2002. URL: <https://eprint.iacr.org/2002/140>.
16. D. Cash, E. Kiltz, and V. Shoup. The twin Diffie-Hellman problem and applications. *Journal of Cryptology*, 22(4):470–504, Oct. 2009. doi:10.1007/s00145-009-9041-6.
17. C. Gentry, P. MacKenzie, and Z. Ramzan. A method for making password-based key exchange resilient to server compromise. In C. Dwork, editor, *Advances in Cryptology – CRYPTO 2006*, volume 4117 of *Lecture Notes in Computer Science*, pages 142–159. Springer, Berlin, Heidelberg, Aug. 2006. doi:10.1007/11818175_9.
18. A. Groce and J. Katz. A new framework for efficient password-based authenticated key exchange. In E. Al-Shaer, A. D. Keromytis, and V. Shmatikov, editors, *ACM CCS 2010: 17th Conference on Computer and Communications Security*, pages 516–525. ACM Press, Oct. 2010. doi:10.1145/1866307.1866365.
19. Y. Gu, S. Jarecki, and H. Krawczyk. KHAPE: Asymmetric PAKE from key-hiding key exchange. In T. Malkin and C. Peikert, editors, *Advances in Cryptology – CRYPTO 2021, Part IV*, volume 12828 of *Lecture Notes in Computer Science*, pages 701–730, Virtual Event, Aug. 2021. Springer, Cham. doi:10.1007/978-3-030-84259-8_24.

20. B. Haase. Security analysis of CPace and (strong) AuCPace, 2020. URL: https://github.com/BjoernMHaase/AuCPace/blob/master/aucpace_security_analysis_20200208.pdf. Accessed 15 Jan. 2025.
21. B. Haase and B. Labrique. AuCPace: Efficient verifier-based PAKE protocol tailored for the IIoT. Cryptology ePrint Archive, Report 2018/286, 2018. URL: <https://eprint.iacr.org/2018/286>.
22. B. Haase and B. Labrique. AuCPace: Efficient verifier-based PAKE protocol tailored for the IIoT. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2019(2):1–48, 2019. URL: <https://tches.iacr.org/index.php/TCHES/article/view/7384>, doi:10.13154/tches.v2019.i2.1-48.
23. S. Hanai, K. Tanaka, M. Tezuka, and Y. Yoshida. Universally composable relaxed asymmetric password-authenticated key exchange. In C. Galdi and D. H. Phan, editors, *SCN 2024: 14th International Conference on Security in Communication Networks, Part II*, volume 14974 of *Lecture Notes in Computer Science*, pages 272–293. Springer, Cham, Sept. 2024. doi:10.1007/978-3-031-71073-5_13.
24. J. Hesse. Separating standard and asymmetric password-authenticated key exchange. Cryptology ePrint Archive, Report 2019/1064, 2019. URL: <https://eprint.iacr.org/2019/1064>.
25. J. Hesse. Separating symmetric and asymmetric password-authenticated key exchange. In C. Galdi and V. Kolesnikov, editors, *SCN 20: 12th International Conference on Security in Communication Networks*, volume 12238 of *Lecture Notes in Computer Science*, pages 579–599. Springer, Cham, Sept. 2020. doi:10.1007/978-3-030-57990-6_29.
26. D. Hofheinz and J. Müller-Quade. Universally composable commitments using random oracles. In M. Naor, editor, *TCC 2004: 1st Theory of Cryptography Conference*, volume 2951 of *Lecture Notes in Computer Science*, pages 58–76. Springer, Berlin, Heidelberg, Feb. 2004. doi:10.1007/978-3-540-24638-1_4.
27. J. Y. Hwang, S. Jarecki, T. Kwon, J. Lee, J. S. Shin, and J. Xu. Round-reduced modular construction of asymmetric password-authenticated key exchange. In D. Catalano and R. De Prisco, editors, *SCN 2018: 11th International Conference on Security in Communication Networks*, volume 11035 of *Lecture Notes in Computer Science*, pages 485–504. Springer, Cham, Sept. 2018. doi:10.1007/978-3-319-98113-0_26.
28. D. P. Jablon. Extended password key exchange protocols immune to dictionary attacks. In *WETICE 1997: 6th IEEE International Workshops on Enabling Technologies Infrastructure for Collaborative Enterprises*, pages 248–255, Cambridge, MA, USA, Jun. 1997. doi:10.1109/ENABL.1997.630822.
29. S. Jarecki, H. Krawczyk, and J. Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. Cryptology ePrint Archive, Report 2018/163, 2018. URL: <https://eprint.iacr.org/2018/163>.
30. S. Jarecki, H. Krawczyk, and J. Xu. OPAQUE: An asymmetric PAKE protocol secure against pre-computation attacks. In J. B. Nielsen and V. Rijmen, editors, *Advances in Cryptology – EUROCRYPT 2018, Part III*, volume 10822 of *Lecture Notes in Computer Science*, pages 456–486. Springer, Cham, Apr. / May 2018. doi:10.1007/978-3-319-78372-7_15.
31. X. Liu, S. Liu, S. Han, and D. Gu. EKE meets tight security in the Universally Composable framework. In A. Boldyreva and V. Kolesnikov, editors, *PKC 2023: 26th International Conference on Theory and Practice of Public Key Cryptography, Part I*, volume 13940 of *Lecture Notes in Computer Science*, pages 685–713. Springer, Cham, May 2023. doi:10.1007/978-3-031-31368-4_24.

32. X. Liu, S. Liu, S. Han, and D. Gu. EKE meets tight security in the universally composable framework. Cryptology ePrint Archive, Report 2023/170, 2023. URL: <https://eprint.iacr.org/2023/170>.
33. D. Pointcheval and G. Wang. VTBPEKE: Verifier-based two-basis password exponential key exchange. In R. Karri, O. Sinanoglu, A.-R. Sadeghi, and X. Yi, editors, *ASIACCS 2017: 12th ACM Symposium on Information, Computer and Communications Security*, pages 301–312. ACM Press, Apr. 2017. doi:10.1145/3052973.3053026.
34. B. F. D. Santos, Y. Gu, S. Jarecki, and H. Krawczyk. Asymmetric PAKE with low computation and communication. In O. Dunkelman and S. Dziembowski, editors, *Advances in Cryptology – EUROCRYPT 2022, Part II*, volume 13276 of *Lecture Notes in Computer Science*, pages 127–156. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07085-3_5.
35. S. Shin, K. Kobara, and H. Imai. Security proof of AugPAKE. Cryptology ePrint Archive, Report 2010/334, 2010. URL: <https://eprint.iacr.org/2010/334>.
36. V. Shoup. Security analysis of SPAKE2+. In R. Pass and K. Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part III*, volume 12552 of *Lecture Notes in Computer Science*, pages 31–60. Springer, Cham, Nov. 2020. doi:10.1007/978-3-030-64381-2_2.
37. T. Taubert and C. A. Wood. SPAKE2+, an Augmented Password-Authenticated Key Exchange (PAKE) Protocol. RFC 9383, Sept. 2023. URL: <https://www.rfc-editor.org/info/rfc9383>, doi:10.17487/RFC9383.

A Proof of Theorem 1

We give the construction of the simulator in the ideal world in Figures 11, 12, and 13. We illustrate the indistinguishability of the real world and the ideal world by constructing some game hoppings. We construct the game hoppings based on the proof of Theorem 1 in [1,2].

Game 0. This is the execution of SPAKE2+ in the real world.

$$\Pr[\mathbf{Real} = 1] = \Pr[\mathbf{Game\ 0} = 1]$$

Game 1. We introduce a simulator $\mathcal{S}$ and an ideal functionality $\mathcal{F}$. In this game, $\mathcal{S}$ simulates everything in the real world. $\mathcal{S}$ completely simulates the functionality of the random oracle $\mathcal{F}_{\text{RO}}$ and records queries to the random oracles H_1 , and H_2 . $\mathcal{S}$ also simulates the common reference string by generating $u, v \leftarrow_{\text{R}} \mathbb{Z}_p$ and setting $U \leftarrow g^u, V \leftarrow g^v$. $\mathcal{F}$ passes all messages outputted by $\mathcal{S}$ and $\mathcal{Z}$. The change from Game 0 to Game 1 does not affect the view of the environment.

$$\Pr[\mathbf{Game\ 0} = 1] = \Pr[\mathbf{Game\ 1} = 1]$$

Game 2. In this game, $\mathcal{S}$ aborts the simulation if $\mathcal{S}$ generates a randomly generated output (π_0, π_1) of random oracle H_2 , and π_0 colludes a value already generated as the first part in an output of H_1 . The random oracle H_1 samples the first part of its output from $\mathbb{Z}_p$ and p is assumed to be large. So, the probability of the environment observing the difference between this game and Game 1 has an upper bound $q_{H_1}^2/p$ and the bound is negligible.

$$|\Pr[\mathbf{Game\ 1} = 1] - \Pr[\mathbf{Game\ 2} = 1]| \leq q_{H_1}^2/p$$

Game 3. In this game, if the adversary does not interfere with protocol flows, the session keys outputted by the parties are not linked to the output of the random oracle H_2 . If a party and the counterpart party use a common password, the parties output a common randomly sampled key. Otherwise, the parties each output a session key taken at random. This game change is the same as Game 2 in the proof of Theorem 1 in [1,2].

The environment observes this change between Game 2 and Game 3 only in the situation where the adversary makes a query $H_2(sid, P_C, P_S, \pi_0, X = g^x M^{\pi_0}, Y = g^y N^{\pi_0}, Z = g^{xy}, N = g^{y\pi_1})$, where $X, Y, (\pi_0, \pi_1)$ coincides with the values used by the parties. We can construct a reduction $\mathcal{B}_1$ that reduces this event to the GCDH assumption. $\mathcal{B}_1$ is the same as the reduction shown in Game 2 in the proof of Theorem 1 in [1,2]. Intuitively, a GCDH challenge (g^{a_1}, g^{a_2}) is embedded in $(X/M^{\pi_0}, Y/N^{\pi_0})$. If the adversary makes a query to the random oracle H_2 which is the pre-image of the session key, then $g^{a_1 a_2} = Z$.

$$|\Pr[\mathbf{Game\ 2} = 1] - \Pr[\mathbf{Game\ 3} = 1]| \leq \text{Adv}_{\mathcal{B}_1}^{\text{GCDH}}$$

Game 4. In this game, $\mathcal{S}$ simulates the message flows of SPAKE2+ without using passwords. When a client is initialized, $\mathcal{S}$ samples random values $x^* \leftarrow \mathbb{Z}_p$ and sends U^{x^*} to the adversary as the message of the client. When a server is initialized, $\mathcal{S}$ samples random values $y^* \leftarrow \mathbb{Z}_p$ and sends V^{y^*} to the adversary as the message of the server. This game change is the same as Game 3 in the proof of Theorem 1 in [1,2].

The simulator in this game uses passwords only when the adversary finds a session key in interfered sessions by querying to the random oracle H_2 . On the client side, the simulator uses $(Y'/V^{\pi_0})^{ux^*-u\pi_0}$ as Z and $(Y'/V^{\pi_0})^{\pi_1}$ as N when checking the correctness of the random oracle query (Y' is a message sent by the adversary to the client.). On the server side, the simulator uses $(X'/U^{\pi_0})^{vy^*-v\pi_0}$ as Z and $(g^{\pi_1})^{vy^*-v\pi_0}$ as N (X' is a message sent by the adversary to the server.). These changes do not affect the view of $\mathcal{Z}$ because the distributions of the messages are not changed and the key generation is simulated as in Game 3.

$$\Pr[\text{Game 3} = 1] = \Pr[\text{Game 4} = 1]$$

Game 5. In this game, $\mathcal{F}$ introduces the RegisterTest, LateTestPwd, TestPwd, Impersonate, and NewKey interfaces. And more, $\mathcal{F}$ memories passwords of parties. If the adversary interferes with them, $\mathcal{S}$ detects queries to the random oracle H_2 made by the adversary corresponds to password guesses on the party, and simulates the key generations using these interfaces.

We illustrate the indistinguishability of this game and Game 4 by separating this game into two parts. The description of this game change is based on Game 4 in the proof of Theorem 1 in [1,2].

Game 5-1. Client side. In this game, we introduce these interfaces on the client side. Consider a situation where the adversary impersonates a server and sends Y' to the client. In this situation, the adversary may find the session key of the client. If the adversary uses the password of the client, the adversary can get the session key of the client by sending a query to the random oracle H_2 . The adversary can make the query either after or before the session is completed. In addition, if the adversary impersonating the server uses the password file stored in the server, and the client and the server use a common password, the adversary can get the session key of the client.

What changes in this game from Game 4 is when the adversary sends the message Y' and the query to H_2 . For the details of this change, we refer to the procedures for receiving a message (sid, Y') and a query H_2 in the simulator.

When the adversary sends Y' , the simulator checks whether the adversary already makes a query to H_2 which is based on a password pw , Y' , and X , where X is the output by the client. If the query is found, the simulator sends a TestPwd query with pw to $\mathcal{F}_{\text{le-aPAKE}}$. When the response is “correct guess”, the simulator sends a NewKey query with the hash of the query. If the query is not found, the simulator checks to see if the adversary is in a situation where the adversary can successfully imitate the server using the password file by checking whether the adversary has already stolen the password file and whether the client and

the server use a common password. If this check passes, the simulator sends an **Impersonate** query to $\mathcal{F}_{\text{le-aPAKE}}$, which allows the simulator to specify the session key of the server. The simulator also sends a **NewKey** query with a randomly taken session key and registers the session key as the response of the query to H_2 which coincides with the session key of the client. When the simulator does not send any query to $\mathcal{F}_{\text{le-aPAKE}}$ in the above procedures, the simulator sends a **RegisterTest** query and **NewKey** query to $\mathcal{F}_{\text{le-aPAKE}}$.

When the adversary sends the query to the hash H_2 , the simulator checks whether there is a password pw of input in H_1 whose first part is equal to π_0 in the query. If the password is found, the simulator checks the correctness of Z and N using x taken when simulating the client and $(\pi_0, \pi_1) = H_1(pw)$. If the check passes, the simulator verifies that the query matches the information of the finished session by checking whether the session is completed and whether Y in the query is equal to Y' which is sent by the adversary. If the verification passes, the simulator sends a **LateTestPwd** query with pw to $\mathcal{F}_{\text{le-aPAKE}}$ and passes the response from $\mathcal{F}_{\text{le-aPAKE}}$ to the adversary. If the simulator does not send any hash value in the above procedures, the simulator sends a randomly taken value to the adversary.

The environment detects the changes between Game 4 and Game 5-1 in three cases. The first case is that the adversary makes more than one password guess in one session. This case can be reduced to the GDL problem and the GCDH problem in the same way as the description in Game 4 of Theorem 1 in [1,2]. Intuitively, in the proof of the theorem, a reduction $\mathcal{B}_2$ embeds a problem instance of the GCDH problem (g^{a_1}, g^{a_2}) into the common reference strings as $U = g^{a_1}, V = g^{a_2}$. In the session where a client sends $X = U^{x^*}$ and the adversary sends Y , if the adversary makes the two hash queries which are considered as the password guesses, the reduction gains (π_0, Z) and (π'_0, Z') which is $Z = \text{CDH}(U^{x^* - \pi_0}, Y/V^{\pi_0})$ and $Z' = \text{CDH}(U^{x^* - \pi'_0}, Y/V^{\pi'_0})$. Then, $(Z^{(x^* - \pi'_0)} Z'^{(x^* - \pi_0)})^{1/(x^* - \pi_0)(x^* - \pi'_0)(\pi_0 - \pi'_0)} = g^{a_1 a_2}$. The detailed calculation is available at Lemma 1 in [1,2]. The probability of happening the situation where $x^* = \pi_0$ or $x^* = \pi'_0$ and the adversary makes the two queries can be reduced to the GDL problem. A reduction $\mathcal{B}_3$ receives a GDL challenge g^w and sets $X = g^w g^t$ by taking t randomly for each session. If the adversary makes the two queries and $X = U^{\pi_0} = g^{u\pi_0}$, the reduction gains $w = u \cdot \pi_0 - t$. The case for π'_0 can be treated in the same way. This reduction for the GDL problem is depicted in Game 4 of Theorem 1 in [1,2].

The second case is that the adversary makes the query to the random oracle H_2 on the session where the adversary impersonates the server and steals the password file after the session is completed. This case is the special case we consider in Section 4.2 and we can construct a reduction $\mathcal{B}_4$ that reduces this case to the GPCCDH problem.

The third case is that the adversary sends a hash query to H_2 with the hash of the password used by the client without querying the password to the random oracle H_1 . After the adversary gets the hash value of H_2 , the adversary can detect the difference of the games by querying the password of the client to

H_1 and comparing the hash value of H_2 and the session key of the client. The adversary has no information about the hash of the password before making the hash query to H_2 , so the probability of this case happening has an upper bound q_{H_2}/p , where q_{H_2} is the number of queries to H_2 , and this is negligible.

By the above discussion, the following relation is shown:

$$\begin{aligned} |\Pr[\text{Game 4} = 1] - \Pr[\text{Game 5-1} = 1]| &\leq \text{Adv}_{\mathcal{B}_2}^{\text{GCDH}} + \text{Adv}_{\mathcal{B}_3}^{\text{GDL}} \\ &\quad + N_C \cdot \text{Adv}_{\mathcal{B}_4}^{\text{GPCDH}} + q_{H_2}/p \end{aligned}$$

where N_C is the number of sessions where a client is initiated.

Game 5-2. Server side. The simulator detects if queries made by the adversary correspond to password guesses on the attacked server instances. The adversary makes a password guess of the client who outputs $Y = g^y$ and receives X' from the adversary by making a query $(\pi_0^*, X', Y, Z' = (X'/U^{\pi_0^*})^y, N' = (X'/U^{\pi_0^*})^{\pi_1^*})$ to H_2 . On the server side, the simulator process as well as the description of the client side in Game 5 except for the attack using the password file. If the adversary uses the password file and the correct random oracle query, the simulator aborts this simulation.

The environment detects the changes between Game 5-1 and Game 5-2 in three cases. The first is that the adversary makes more than one password guess in one session. We can construct a reduction $\mathcal{B}_5$ that reduces this case to the GCDH problem and a reduction $\mathcal{B}_6$ that reduces a part of this case to the GDL problem in the same way as the first case of Game 5-1.

The second is the case where the adversary makes the correct hash query to H_2 after the adversary steals the password file and before the adversary queries the password of the server to H_1 . We can construct a reduction $\mathcal{B}_7$ that reduces this case to the GCDH problem in the same way as Theorem 3 in [36]. We described this reduction in Section 4.2.

The third case is that the adversary sends a hash query to the random oracle H_2 with the hash of the password used by the client without querying the password to the random oracle H_1 . The explanation of this case is the same as for the explanation of the third case of Game 5-1.

The above discussion leads to the following inequality:

$$\begin{aligned} |\Pr[\text{Game 5-1} = 1] - \Pr[\text{Game 5-2} = 1]| &\leq \text{Adv}_{\mathcal{B}_5}^{\text{GCDH}} + \text{Adv}_{\mathcal{B}_6}^{\text{GDL}} \\ &\quad + N_{s-c} \cdot \text{Adv}_{\mathcal{B}_7}^{\text{GCDH}} + q_{H_2}/p \end{aligned}$$

where N_{s-c} is the number of client-server pairs.

Game 6. In this game, we change the structure of $\mathcal{F}$ to $\mathcal{F}_{\text{le-aPAKE}}$. $\mathcal{F}$ introduces StorePwdFile, StealPwdFile, SvrSession, CltSession, and OfflineTestPwd interfaces. The SvrSession messages and CltSession messages allow the simulator to know when clients or servers send messages to execute the protocol.

When the adversary steals the password file of a server, the simulator sends a StealPwdFile query to $\mathcal{F}_{\text{le-aPAKE}}$. If the response from $\mathcal{F}_{\text{le-aPAKE}}$ is pw , the simulator

searches $H_1(pw) = (\pi_0, \pi_1)$ and sends (π_0, g^{π_1}) to the adversary. Else if the response is “password file stolen”, the simulator generates $(\pi_0^*, \pi_1^*) \leftarrow_{\mathbb{R}} \mathbb{Z}_p^2$ and sends $(\pi_0^*, g^{\pi_1^*})$ to the adversary.

When the adversary makes a query to the random oracle H_1 , the simulator makes an **OfflineTestPwd** query with the password in the oracle query. Based on the response, the simulator coordinates the response from the random oracle H_1 to the adversary with the password file. If the response is “correct guess”, the simulator searches the password file of the server (π_0, g^{π_1}) and sends (π_0, π_1) to the adversary. Otherwise, the simulator sends randomly taken hash value to the adversary.

The change between this game and Game 5 does not affect the view of the environment. The behavior of the simulator in this game coincides with the simulator shown in Figures 11, 12, and 13. So, this game is equal to the ideal world.

$$\Pr[\mathbf{Game\ 5} = 1] = \Pr[\mathbf{Game\ 6} = 1] = \Pr[\mathbf{IDEAL} = 1]$$

Proof that SPAKE2+ realizes $\mathcal{F}_{\text{w-le-aPAKE}}$. Since w-le-aPAKE is weaker than le-aPAKE, the proof that SPAKE2+ realizes $\mathcal{F}_{\text{w-le-aPAKE}}$ follows from Theorem 1. However, we describe the proof by modifying the proof of Theorem 1 to clarify the relationship between SPAKE2+ and the difference between $\mathcal{F}_{\text{w-le-aPAKE}}$ and $\mathcal{F}_{\text{le-aPAKE}}$.

We make two changes to the construction of the simulator. First, we change the procedure where the adversary sends (sid, Y) to the client. We make the simulator in this proof not send an **Impersonate** query when the password file is compromised. Concretely, the simulator in this proof ignores the third branch of the procedure on receiving (sid, Y) of the simulator in Theorem 1. This change makes the simulator independent of the D values in **ClSession** messages.

Second, we change the procedure in which the adversary makes a query to the random oracle H_2 . When the adversary makes a query which equal to the session key of the client in a session where the password file is compromised, the simulator sends a **LateTestPwd** query with $\perp$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ and sends the session key received from $\mathcal{F}_{\text{w-le-aPAKE}}$ to the adversary. Concretely, we change the fifth branch of the procedure on receiving an H_2 query.

The difference of the game hoppings between this theorem and Theorem 1 is Game 5-1. In this game, the simulator uses a **LateTestPwd** query with $\perp$ when the adversary that compromises a server makes a query to H_2 which coincides the session key of the client that is the counterpart of the server. By this change, if the adversary causes the special case mentioned in the proof of Theorem 1, the simulator gets the session key of the client by sending a **LateTestPwd** query with $\perp$ and the environment cannot distinguish this game change. So, in this proof, we do not have to construct a non-tight reduction that reduces the special case to the GPCCDH problem.

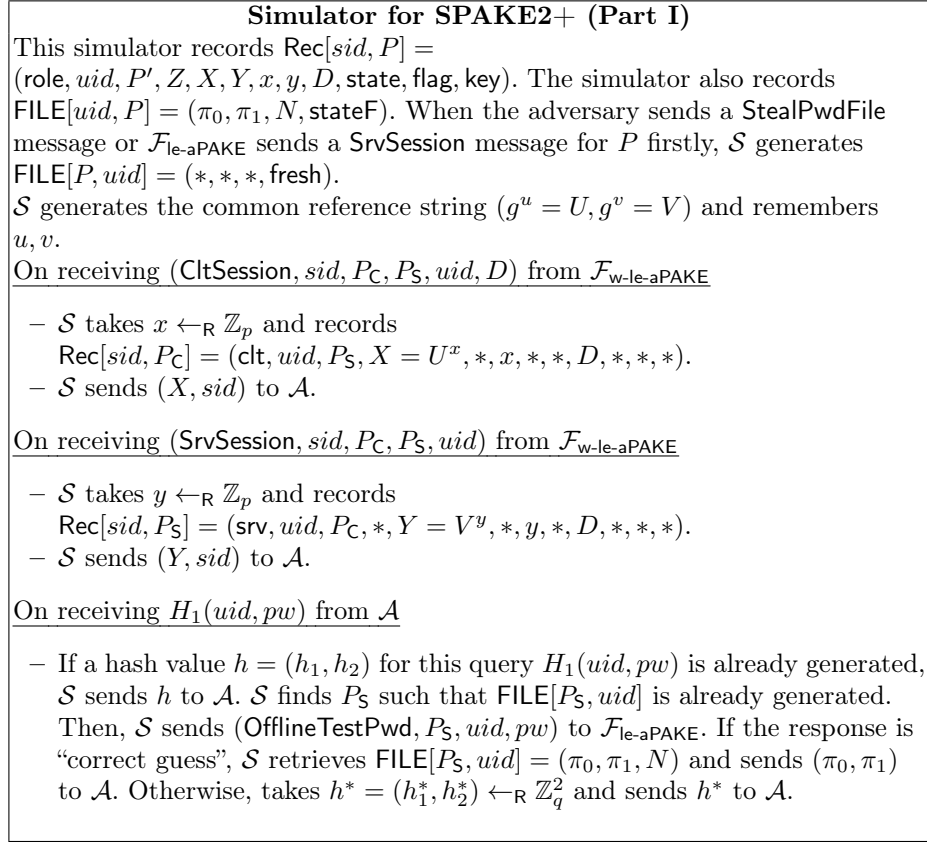


Fig. 11. The simulator for Theorem 1 (Part I). This simulator is based on the simulator for the security proof of SPAKE2 in [1,2].

Simulator for SPAKE2+ (Part II)	
On receiving $H_2(sid, \pi_0, P_C, P_S, X^*, Y^*, Z^*, N^*)$ from $\mathcal{A}$	
– If a hash value h for this query $H_2(sid, \pi_0, P_C, P_S, X^*, Y^*, Z^*, N^*)$ is already generated, $\mathcal{S}$ sends h to $\mathcal{A}$. – $\mathcal{S}$ takes $k \leftarrow_{\mathcal{R}} \{0, 1\}^\kappa$. – If $(X^*, Y^*, Z^*) = (X, Y, (X/U^{\pi_0})^{vy-v\pi_0})$, where X is in $\text{Rec}[P_C, sid]$ and Y, y are in $\text{Rec}[P_S, sid]$, $\mathcal{S}$ aborts this simulation (the discussion in Game 3). – If pw such that $H_1(uid, pw) = (\pi_0, \pi_1)$ exists, • If $(Z^*, N^*) = ((Y^*/V^{\pi_0})^{ux-u\pi_0}, (Y^*/V^{\pi_0})^{\pi_1})$, where x is in $\text{Rec}[P_C, sid]$, * If flag in $\text{Rec}[P_C, sid]$ is tested and X^*, Y^* is X, Y in $\text{Rec}[P_C, sid]$, $\mathcal{S}$ sends $(\text{LateTestPwd}, sid, P_C, P_S, uid, pw)$ to $\mathcal{F}_{\text{le-aPAKE}}$. $\mathcal{S}$ receives the response (sid, k^*) and sets $k \leftarrow k^*$ to $\mathcal{A}$. $\mathcal{S}$ changes flag to unflagged. • If $(Z^*, N^*) = ((X^*/U^{\pi_0})^{y-v\pi_0}, (Y/V^{\pi_0})^{\pi_1})$, where Y, y is in $\text{Rec}[P_S, sid]$, * If flag in $\text{Rec}[P_S, sid]$ is tested and X^*, Y^* is X, Y in $\text{Rec}[P_S, sid]$, $\mathcal{S}$ sends $(\text{LateTestPwd}, sid, P_S, P_C, uid, pw)$ to $\mathcal{F}_{\text{le-aPAKE}}$. $\mathcal{S}$ receives the response (sid, k^*) and sets $k \leftarrow k^*$ to $\mathcal{A}$. $\mathcal{S}$ changes flag to unflagged. – Else if stateF in $\text{FILE}[P_S, uid] = (\pi_0^*, \pi_1^*, *, *)$ is compromised, X^*, Y^* is X, Y in $\text{Rec}[P_C, sid]$, and flag in $\text{Rec}[P_C, sid]$ is tested, $\mathcal{S}$ checks $(Z^*, N^*) = ((Y^*/V^{\pi_0})^{ux-u\pi_0^*}, (Y^*/V^{\pi_0^*})^{\pi_1})$, where x is in $\text{Rec}[P_C, sid]$. If this check passes, $\mathcal{S}$ aborts this simulation (the discussion in Game 5-1). – $\mathcal{S}$ sends k to the adversary.	
On receiving $(\text{StealPwdFile}, P_S, uid)$ from $\mathcal{A}$ to P_S	
– $\mathcal{S}$ sends $(\text{StealPwdFile}, P_S, uid)$ to $\mathcal{F}_{\text{le-aPAKE}}$. If the response is “no password file”, $\mathcal{S}$ sends “no password file” to $\mathcal{A}$. If the response is pw, $\mathcal{S}$ finds $(\pi_0, \pi_1) = H_1(uid, pw)$, sends $(\pi_0, N = g^{\pi_1})$ (this hash value must be already generated.), and stores $\text{FILE}[P_S, uid] = (\pi_0, N, \pi_1, \text{compromised})$. If the response is “password file stolen”, $\mathcal{S}$ takes $(\pi_0, \pi_1) \leftarrow_{\mathcal{R}} \mathbb{Z}_p^2$ and stores $\text{FILE}[P_S, uid] = (\pi_0, N = g^{\pi_1}, \pi_1, \text{compromised})$. $\mathcal{S}$ sends (π_0, N) to $\mathcal{A}$.	

Fig. 12. The simulator for Theorem 1 (Part II).

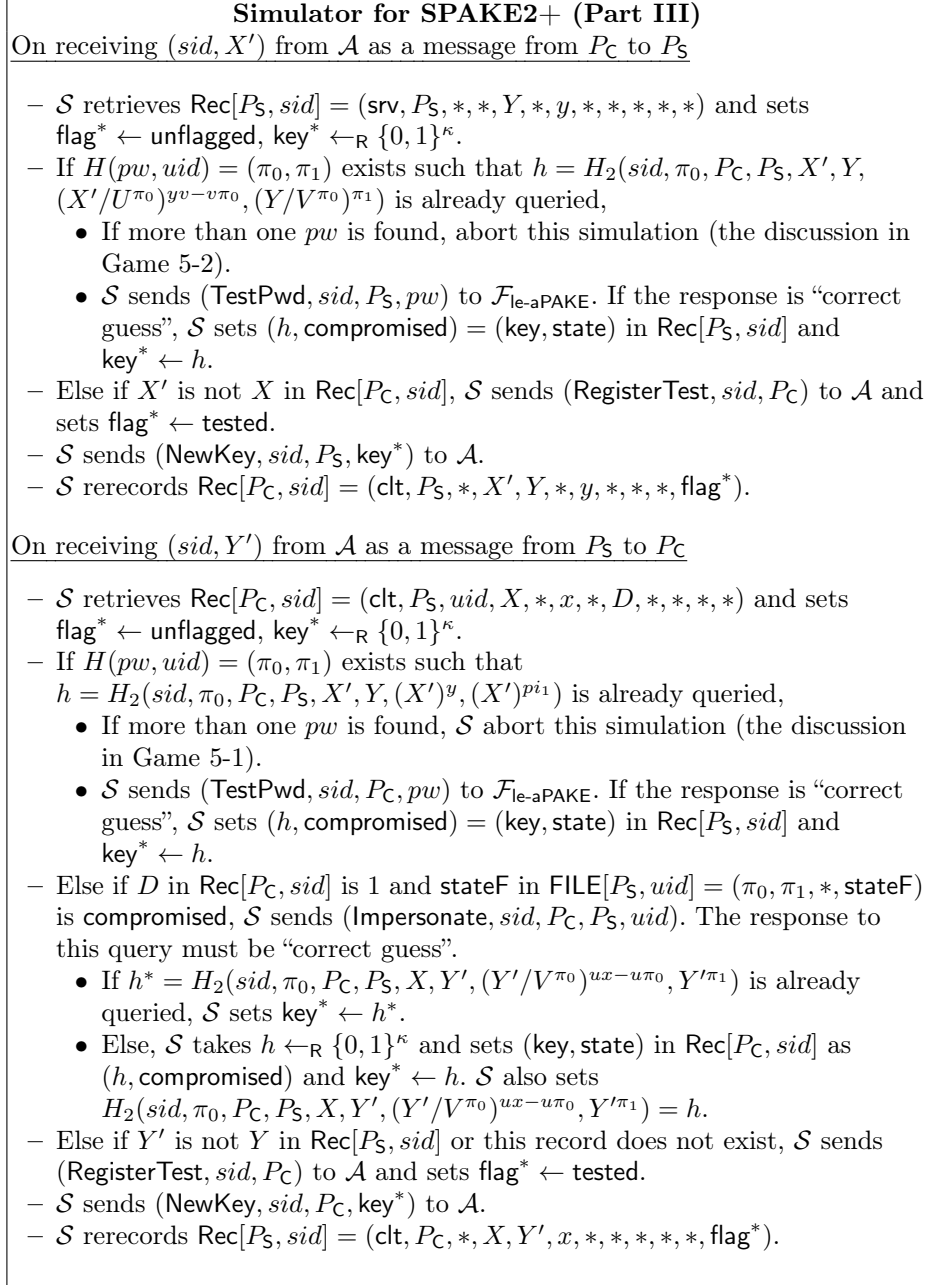


Fig. 13. The simulator for Theorem 1 (Part III).

B Proof of Theorem 2

In Figures 14, 15, and 16, we give the construction of the simulator in the ideal world. We prove the indistinguishability of the real world and the ideal world by giving some games and showing the indistinguishability of these games.

Game 0. This game is the same as the real world. So, the view of the environment is not changed from the real world.

$$\Pr[\mathbf{REAL} = 1] = \Pr[\mathbf{Game\ 0} = 1]$$

Game 1. In this game, the simulator $\mathcal{S}$ simulates parties for the adversary by viewing the inputs to the parties. We introduce $\mathcal{F}$ as an interface between the environment and $\mathcal{S}$. In this game, $\mathcal{F}$ does not have any function and only passes all of the messages between the environment and $\mathcal{S}$. $\mathcal{S}$ has sufficient information for simulating the real world, so the adversary cannot distinguish this game and Game 0.

$$\Pr[\mathbf{Game\ 0} = 1] = \Pr[\mathbf{Game\ 1} = 1]$$

Game 2. In this game, the simulator $\mathcal{S}$ aborts the simulation if the random oracle H outputs a hash value that collides with another output that the random oracle H has already outputted. The random oracle H takes outputs from $\mathbb{Z}_p$, so the probability of this collision happening has an upper bound q_H^2/p , where q_H is the number of random oracle queries and is negligible because p is enough large.

$$|\Pr[\mathbf{Game\ 1} = 1] - \Pr[\mathbf{Game\ 2} = 1]| \leq q_H^2/p$$

Game 3. In this game, we introduce a modified **NewKey** query in $\mathcal{F}$. When searching states of parties, **fresh**, **completed**, **compromised**, and **interrupted**, $\mathcal{F}$ looks at the states of the parties in $\mathcal{F}_{\text{lePAKE}}$. When the adversary sends a **NewKey** query for a party P in $\mathcal{F}_{\text{lePAKE}}$, the simulator $\mathcal{S}$ checks whether P and its partner P' use the same X value (in other words, the message (sid, X) output by the party in the server role is received by the party on the client role without being changed.). If they use the same X value, the simulator passes the **NewKey** query to $\mathcal{F}$. Otherwise, the simulator sends a **TestPwd** query with 0 to $\mathcal{F}_{\text{lePAKE}}$ and the **NewKey** query sent by the adversary to $\mathcal{F}$. This game and Game 2 are identical in terms of the view of the environment.

$$\Pr[\mathbf{Game\ 2} = 1] = \Pr[\mathbf{Game\ 3} = 1]$$

Game 4. In this game, $\mathcal{F}$ introduces **LateTestPwd** and **Impersonate** interface. These interfaces are defined as well as the interfaces in $\mathcal{F}_{\text{w-le-aPAKE}}$. By using these interfaces, the simulator simulates the responses to **TestPwd** queries and **LateTestPwd** queries made by the adversary without seeing the passwords of the

parties. The usage of these interfaces is the same as in the simulator described in Section 5.2, so we repeat the same explanation. When receiving a **TestPwd** query or a **LateTestPwd** query with (W', X', Z') for a party who uses X' , $\mathcal{S}$ searches pw such that $Z' = X'^{H(uid, salt, \sigma, pw)}$ and $W' = g^{H(uid, salt, \sigma, pw)}$ in the record of the random oracle H .

- If pw is found, $\mathcal{S}$ sends $(\text{TestPwd}, sid, P, pw)$ or $(\text{LateTestPwd}, sid, P, P', uid, pw)$ to $\mathcal{F}_{w\text{-le-aPAKE}}$, and passes the response from $\mathcal{F}_{w\text{-le-aPAKE}}$ to $\mathcal{A}$. $\mathcal{S}$ uses the same type of query that $\mathcal{A}$ uses.
- If not, $\mathcal{S}$ examines whether the password file $(\text{FILE}, salt, \sigma, W = g^w)$ is used to calculate Z' by checking whether the role of P is the client, the password file of the session is stolen, and (Z', W') equals (X'^w, W) .
 - If the result of this check is true, $\mathcal{S}$ sends $(\text{Impersonate}, sid, P, P', uid)$ or $(\text{LateTestPwd}, sid, P, P', uid, \perp)$ to $\mathcal{F}_{w\text{-le-aPAKE}}$ and passes the response from $\mathcal{F}_{w\text{-le-aPAKE}}$ to $\mathcal{A}$. If $\mathcal{A}$ uses the **TestPwd** query, $\mathcal{S}$ uses the **Impersonate** query. Otherwise, $\mathcal{S}$ uses the **LateTestPwd** query.
 - Otherwise, $\mathcal{S}$ sends $(\text{TestPwd}, sid, P, 0)$ or $(\text{LateTestPwd}, sid, P, P', uid, 0)$ to $\mathcal{F}_{w\text{-le-aPAKE}}$ and passes the response from $\mathcal{F}_{w\text{-le-aPAKE}}$ to $\mathcal{A}$. $\mathcal{S}$ uses the same type of query that $\mathcal{A}$ uses.

The environment can detect the changes between this game and Game 3 in two cases. The first is the case where the adversary guesses (X^w, X, W) on the server side by stealing the password file W and not querying the password of the server to the random oracle H . We can construct a reduction $\mathcal{B}_{\text{GCDH}}$ that reduces this case to the GCDH problem. We have already described the construction of this reduction in Section 5.2.

We note that if the adversary passes the first message from a server to a client without changing it, then the adversary cannot guess the tuple used as a password of $\mathcal{F}_{\text{lePAKE}}$ on the client side by only stealing W in the password file. However, the simulator can send the **Impersonate** query or **LateTestPwd** query with $\perp$ to $\mathcal{F}$ and can select or know the session key of the client. Therefore, this security feature of the le-AuCPace is not captured by the $\mathcal{F}_{w\text{-le-aPAKE}}$,

The second is the case where the adversary queries a **TestPwd** query or a **LateTestPwd** query whose parameter is based on the hash of the password of the party without querying the password to H . After this, if the adversary queries the hash of the password and the environment checks whether the response to the **TestPwd** or **LateTestPwd** is equal to the session key of the party, the environment observes this game change. The adversary has no information about the hash value, so the probability of this case happening has an upper bound q/p , where q is the number of queries of **TestPwd** and **LateTestPwd**, and this bound is negligible.

Because of the descriptions for the two cases, the environment cannot distinguish this game and Game 3.

$$|\Pr[\mathbf{Game\ 3} = 1] - \Pr[\mathbf{Game\ 4} = 1]| \leq N_{s-c} \cdot \text{Adv}_{\text{GCDH}} + q/p$$

where N_{s-c} is the number of client-server pairs.

Game 5. In this game, $\mathcal{F}$ does not reveal passwords and the simulator simulates protocol executions and the random oracle without seeing passwords and password files. $\mathcal{F}$ introduces the `ClSession`, `SltSession`, `StealPwdFile`, `OfflineTestPwd`, and `StorePwdFile` interfaces. The usage of these interfaces is the same as in the simulator described in Section 5.2, so we repeat the same explanation.

The simulator $\mathcal{S}$ simulates the behavior of clients and servers, remembering $(salt, \sigma, X)$ used by each party and the password file of the servers. For a client, $\mathcal{S}$ sends a `username` message to the adversary $\mathcal{A}$ when a `ClSession` message of the client is sent by $\mathcal{F}_{w\text{-le-aPAKE}}$. Then, if $\mathcal{A}$ sends a `param` message to the client, $\mathcal{S}$ stores $(salt, \sigma, X)$ in the message and sends a `NewSession` message to $\mathcal{A}$ as a message sent by $\mathcal{F}_{\text{lePAKE}}$. For a server, $\mathcal{S}$ generates $salt, \sigma$ randomly, and stores a password file of the server at the time a `StealPwdFile` message or `SrvSession` message of the server is sent by $\mathcal{F}_{w\text{-le-aPAKE}}$. At a later time, If $\mathcal{S}$ receives a `username` message sent by $\mathcal{A}$ to the server, $\mathcal{S}$ takes $x \leftarrow_R \mathbb{Z}_p$, sends a `param` message including $(salt, \sigma, X = g^x)$, stores $(salt, \sigma, X)$, and sends a `NewSession` message to $\mathcal{A}$ as a message sent by $\mathcal{F}_{\text{lePAKE}}$.

When a `StealPwdFile` query is sent to a server by $\mathcal{A}$, the simulator $\mathcal{S}$ passes this query to $\mathcal{F}_{w\text{-le-aPAKE}}$. If the response from the functionality is pw , then $\mathcal{S}$ retrieves the password file $(\text{FILE}, salt, \sigma, *)$, searches the hash value $w = H(uid, salt, \sigma, pw)$, and sends $(uid, salt, \sigma, W = g^w)$ to $\mathcal{A}$. $\mathcal{S}$ updates the password file to $(\text{FILE}, salt, \sigma, W)$. Otherwise, $\mathcal{S}$ takes $w^* \leftarrow_R \mathbb{Z}_p$, sends $(uid, salt, \sigma, W^* = g^{w^*})$ to $\mathcal{A}$, and stores $(\text{FILE}, salt, \sigma, W^*)$ as the password file of the server.

When the adversary $\mathcal{A}$ queries $(uid, salt, \sigma, pw)$ to the random oracle H , if the hash value of this query is already generated, $\mathcal{S}$ sends it to $\mathcal{A}$. If not, the simulator $\mathcal{S}$ checks whether the server of uid is already initiated by the `StorePwdFile` message and the password file of this server is $(\text{FILE}, salt, \sigma, *)$. If the result of this check is true, $\mathcal{S}$ sends $(\text{OfflineTestPwd}, P_S, uid, pw)$ to $\mathcal{F}_{w\text{-le-aPAKE}}$. If the response from $\mathcal{F}_{w\text{-le-aPAKE}}$ is “correct guess”, $\mathcal{S}$ retrieves the password file $(\text{FILE}, salt, \sigma, W = g^w)$ and returns w to $\mathcal{A}$. If $\mathcal{S}$ does not send a response to $\mathcal{A}$ in the above procedure, $\mathcal{S}$ samples $h \leftarrow_R \mathbb{Z}_p$, returns h to $\mathcal{A}$, and stores h as the hash value of this query.

The view of the environment is not changed from Game 4, so the environment cannot distinguish Game 4 and Game 5. $\mathcal{F}$ in this game is identical to $\mathcal{F}_{w\text{-le-aPAKE}}$ and the behavior of the simulator in this game is identical to the simulator shown in Figures 14, 15, and 16, so Theorem 2 is proven.

$$\Pr[\text{Game 4} = 1] = \Pr[\text{Game 5} = 1] = \Pr[\text{IDEAL} = 1]$$

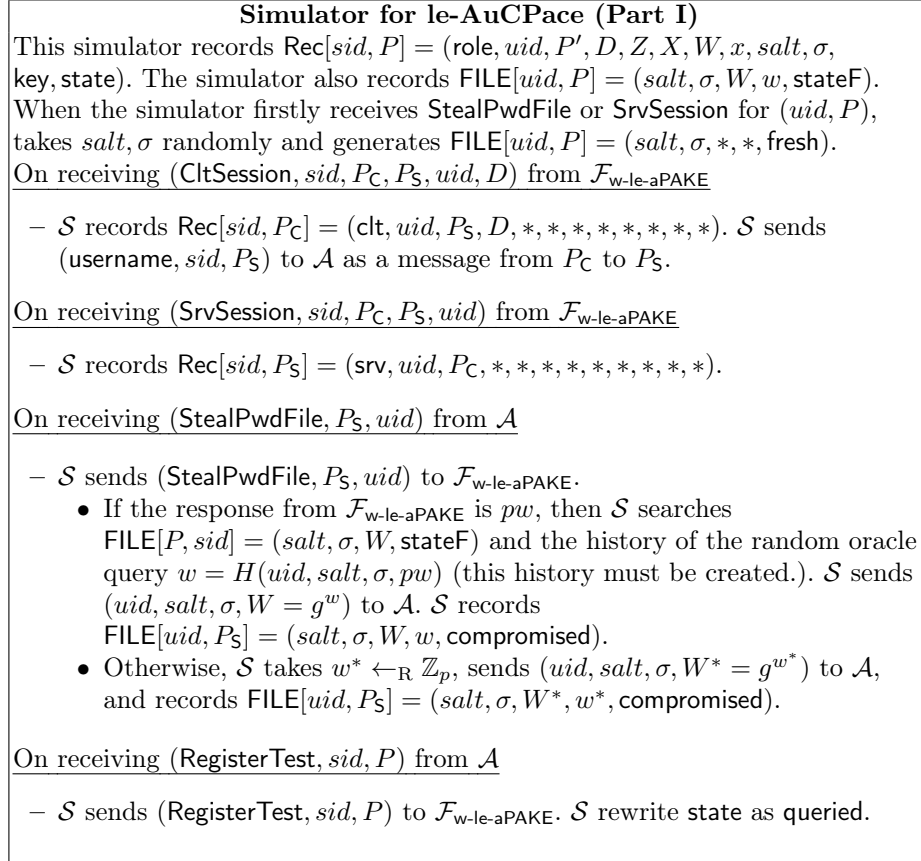
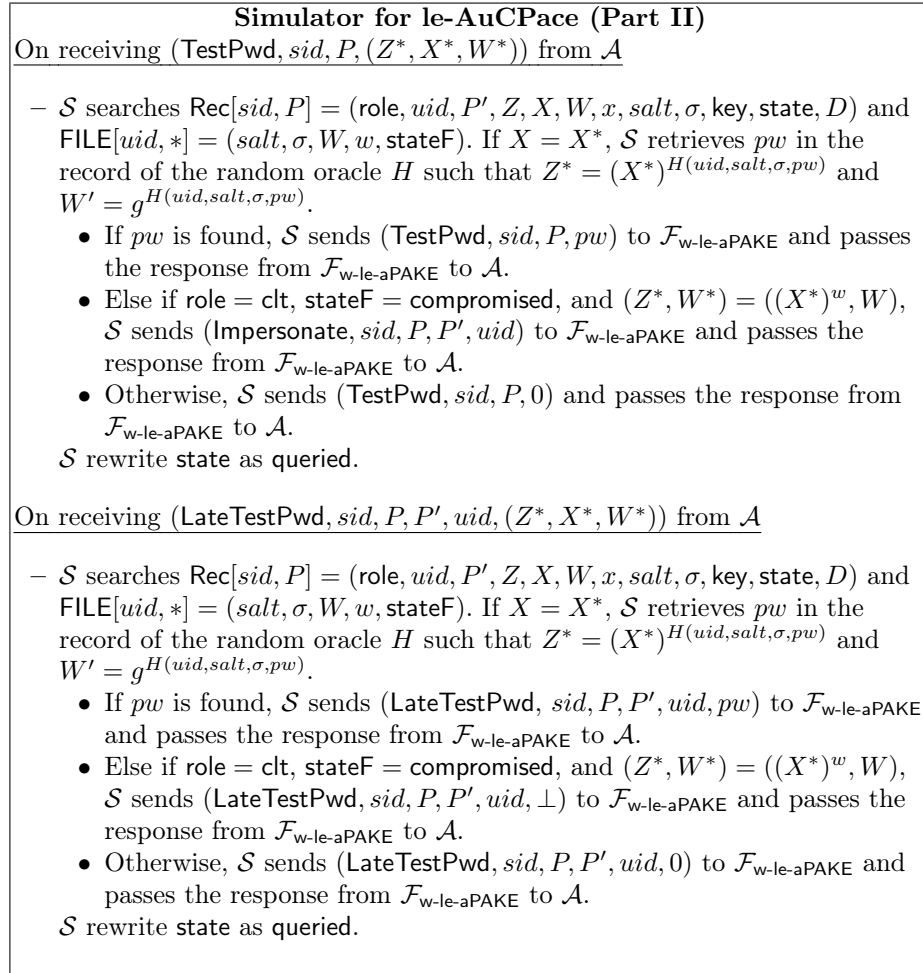


Fig. 14. The simulator for Theorem 2 (Part I). A brief description of this simulator is in Section 5.2. Here, we present this simulator with some additional details.

**Fig. 15.** The simulator for Theorem 2 (Part II).

Simulator for le-AuCPace (Part III)	
<u>On receiving (NewKey, sid, P, k^*) from $\mathcal{A}$</u>	
– $\mathcal{S}$ searches $\text{Rec}[sid, P] = (\text{role}, uid, P', Z, X, W, x, salt, \sigma, \text{key}, \text{state}, D)$ and $\text{Rec}[sid, P'] = (\text{role}, uid, P, Z', X', W', x', salt', \sigma', \text{key}', \text{state}', D')$. • If $\text{state} = \text{state}' = \text{fresh}$ and $X \neq X'$, sends $(\text{TestPwd}, sid, P, 0)$ to $\mathcal{F}_{\text{w-le-aPAKE}}$. $\mathcal{S}$ passes the NewKey query from $\mathcal{A}$ to $\mathcal{F}_{\text{w-le-aPAKE}}$ and the response from $\mathcal{F}_{\text{w-le-aPAKE}}$ to $\mathcal{A}$.	
<u>On receiving $H(uid, salt^*, \sigma^*, pw^*)$ from $\mathcal{A}$</u>	
– If a hash value of this query has been already generated, $\mathcal{S}$ sends the hash value to $\mathcal{A}$. – $\mathcal{S}$ takes $h \leftarrow_{\mathcal{R}} \mathbb{Z}_p$ and retrieves $\text{FILE}[uid, P_S] = (salt, \sigma, W, w, \text{stateF})$. – If $(salt, \sigma) = (salt^*, \sigma^*)$, $\mathcal{S}$ sends $(\text{OfflineTestPwd}, P_S, uid, pw^*)$ to $\mathcal{F}_{\text{w-le-aPAKE}}$. • If the response from $\mathcal{F}_{\text{w-le-aPAKE}}$ is “correct guess”, $\mathcal{S}$ sets $h \leftarrow w$. – $\mathcal{S}$ returns h to $\mathcal{A}$, and stores h as the hash value of this query.	
<u>On receiving (username, sid, P_C) from $\mathcal{A}$ as a message from P_C to P_S</u>	
– $\mathcal{S}$ searches $\text{Rec}[sid, P_S] = (\text{srv}, uid, P_C, *, *, *, *, *, *, *, *)$ and $\text{FILE}[uid, P_S] = (salt, \sigma, W, *)$. $\mathcal{S}$ takes $x \leftarrow_{\mathcal{R}} \mathbb{Z}_p$ and sends $(\text{param}, sid, P_S, salt, \sigma, X = g^x)$ to $\mathcal{A}$ as a message from P_S to P_C. $\mathcal{S}$ also sends $(\text{NewSession}, sid, P_S, P_C, \text{srv})$ to $\mathcal{A}$ as a message from $\mathcal{F}_{\text{lePAKE}}$ to $\mathcal{A}$. $\mathcal{S}$ rerecords $\text{Rec}[sid, P_S] = (\text{srv}, uid, P_C, *, X, *, x, salt, \sigma, *, *)$.	
<u>On receiving (param, $sid, salt', \sigma', X'$) from $\mathcal{A}$ as a message from P_S to P_C</u>	
– $\mathcal{S}$ searches $\text{Rec}[sid, P_C] = (\text{clt}, uid, P_S, *, *, *, *, *, *, *, D)$. $\mathcal{S}$ sends $(\text{NewSession}, sid, P_S, P_C, \text{clt})$ to $\mathcal{A}$ as a message from $\mathcal{F}_{\text{lePAKE}}$ to $\mathcal{A}$. $\mathcal{S}$ rerecords $\text{Rec}[sid, P_C] = (\text{srv}, uid, P_C, *, X', *, *, salt', \sigma', *, *, D)$.	

Fig. 16. The simulator for Theorem 2 (Part III).

C Proof of Theorem 3

We show the proof of this theorem based on the proof of Theorem 2 in [2], which states the security of the compiler from lePAKE to rPAKE-EA. In Figures 17, 18, and 19, we give the simulator in the ideal world for this theorem. We construct this simulator based on the simulator for the compiler from lePAKE to rPAKE-EA shown in Fig.6 of [1,2].

We prove this theorem by adding some discussions to the proof of Theorem 2 in Appendix A of [2]. Firstly, we consider one additional case.

- Case 1' (Online attack on w-le-aPAKE when a server is compromised $\rightarrow$ online attack on r-aPAKE-EA when a server is compromised) The password file of P_S is compromised and the adversary $\mathcal{A}$ sends (**Impersonate**, P_C , P_S , uid) to $\mathcal{F}_{w-le-aPAKE}$ when the state of P_C is fresh. There are two sub-cases:
 - Case 1'a: P_C and P_S use a common password.
 - Case 1'b: This is the case used when the condition necessary for Case 1'a is not satisfied.

In addition, we consider one additional sub-case in Case 2.

- Case 2c: $pw = \perp$, password file is stolen, and the client and the server use a common password.

In defining Game 1, Game 4, and Game 5, we apply the changes defined in Cases 1b, 2b, and 4b to Case 1'b.

Now, we argue that the views of $\mathcal{Z}$ in Game 5, which is the ideal world, on Case 1'a, Case 1'b, and Case 2c are indistinguishable from the views of $\mathcal{Z}$ in the real world.

The description of the simulation of Case 1'a in the ideal world does not differ from the one of Case 1a, except that the **TestPwd** messages sent by the simulator are changed to **Impersonate** messages. Likewise, the simulation of Case 1'b is not different from the one of Case 1b, except that the **TestPwd** messages sent by the simulator are changed to **Impersonate** messages. We have already discussed these two cases in the ideal world in “When the adversary uses **Impersonate** queries.” in the brief explanation of Section 6.

The description of the simulator in Case 2c does not differ from the one in Case 1a except that the message using **TestPwd** is changed to **RegisterTest** and **LateTestPwd**. We have already discussed this case as the situation named “When the adversary uses **RegisterTest** queries” in the brief explanation of Section 6, but we explain this case in detail to clarify how to use **RegisterTest** and **LateTestPwd** in this situation.

Game 5. In this case, the simulations in Game 0 and Game 5 are equal. When the adversary $\mathcal{A}$ sends (**RegisterTest**, sid , P) and (**NewKey**, sid , P , k^*) to P , $\mathcal{A}$ receives a key confirmation message τ . Then when $\mathcal{A}$ sends (**LateTestPwd**, sid , P , P' , uid , $\perp$), $\mathcal{A}$ receives a session key k . If the role of P is a client, $\mathcal{A}$ steals the password file of P' , and P and P' use the same password, $\tau = \text{PRF}_k(2)$ and when $\mathcal{A}$ sends $\text{PRF}_k(3)$ to P , P outputs $\text{PRF}_k(1)$ as the session key. Otherwise, $\tau \neq \text{PRF}_k(2)$ and P outputs **abort** when $\mathcal{A}$ sends $\text{PRF}_k(3)$ to P .

Ideal world. On receiving $(\text{RegisterTest}, sid, P)$ and $(\text{NewKey}, sid, P, k^*)$ from $\mathcal{A}$, the simulator randomly takes k from $\{0, 1\}^\kappa$ and sends $\tau = \text{PRF}_k(2)$. After this, on receiving $(\text{LateTestPwd}, sid, P, P', uid, \perp)$ from $\mathcal{A}$, the simulator verifies that the password file of P' is already stolen and D in $(\text{ClSession}, sid, P, P', uid, D)$, which the simulator has received from $\mathcal{F}_{\text{r-aPAKE-EA}}$, is 1. If the verification passes, the simulator sends k to $\mathcal{A}$. Furthermore, if $\mathcal{A}$ sends $\text{PRF}_k(2)$ to P , the simulator sends $(\text{Impersonate}, sid, P, P', uid)$ and then $(\text{NewKey}, sid, P, \text{PRF}_k(1))$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$. Else, the simulator sends $(\text{NewKey}, sid, P, 0)$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$.

Simulator for Theorem 3 (Part I)

$\mathcal{S}$ stores $\text{Rec}[P, sid] = (\text{role}, P', uid, D, \text{state}, \text{flag}, \text{key}, \text{conf})$. $\mathcal{S}$ also stores $\text{FILE}[P, uid] = (\text{stateF})$. When the simulator receives StealPwdFile or SrvSession for (uid, P) firstly, $\mathcal{S}$ generates $\text{FILE}[P, uid] = (\text{fresh})$.

On receiving $(\text{ClSession}, sid, P_C, P_S, uid, D)$ from $\mathcal{F}_{\text{r-aPAKE-EA}}$

- If this is the first ClSession message for sid , $\mathcal{S}$ sends $(\text{ClSession}, sid, P_C, P_S, uid, D)$ to $\mathcal{A}$ and stores $\text{Rec}[P_C, sid] = (\text{clt}, P_S, uid, D, \text{fresh}, \text{unflagged}, *, *)$.

On receiving $(\text{SrvSession}, sid, P_C, P_S, uid)$ from $\mathcal{F}_{\text{r-aPAKE-EA}}$

- If this is the first SrvSession message for sid , $\mathcal{S}$ sends $(\text{SrvSession}, sid, P_C, P_S, uid)$ to $\mathcal{A}$ and stores $\text{Rec}[P_S, sid] = (\text{Srv}P_C, uid, D, \text{fresh}, \text{unflagged}, *, *)$.

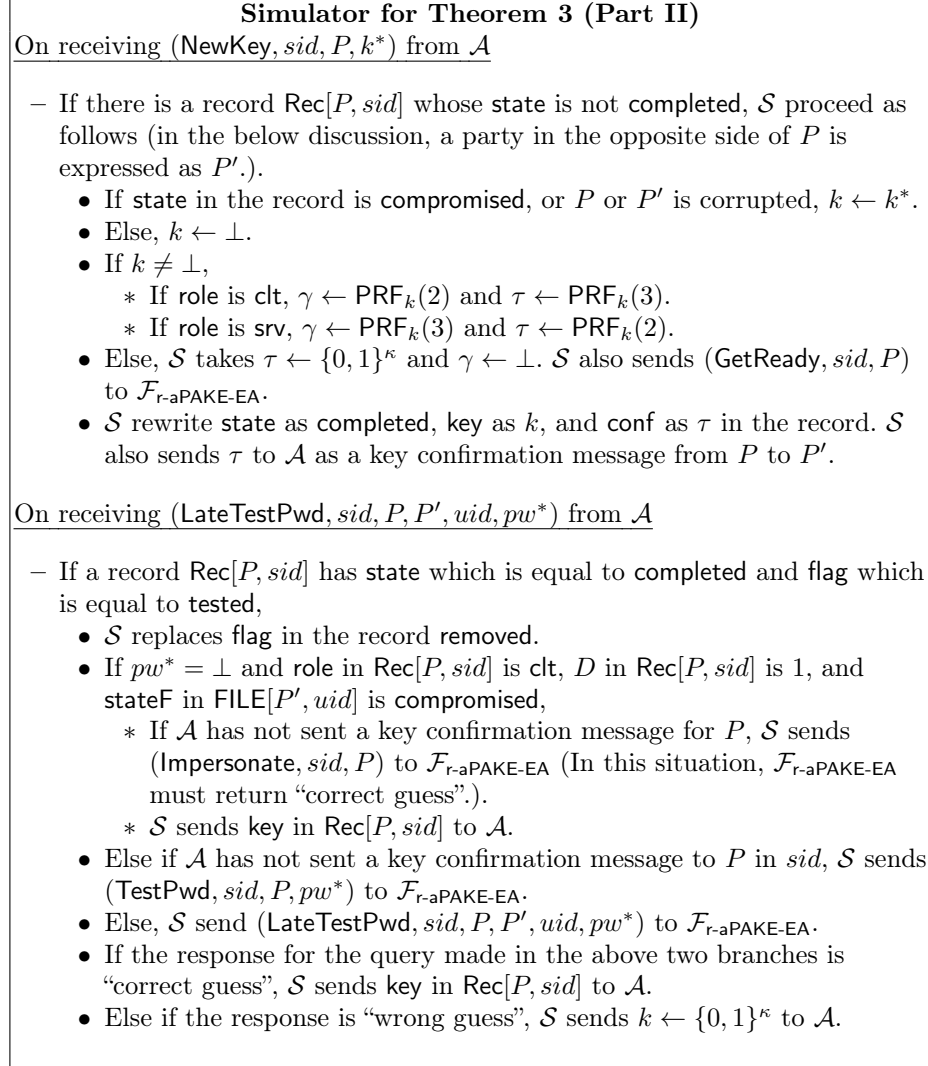
On receiving $(\text{TestPwd}, sid, P, pw)$ from $\mathcal{A}$

- If there is a record $\text{Rec}[P, sid]$ whose state is fresh, $\mathcal{S}$ sends $(\text{TestPwd}, sid, P, pw)$ to $\mathcal{F}_{\text{r-aPAKE-EA}}$.
 - If the response is "correct guess", $\mathcal{S}$ passes the response to $\mathcal{A}$ and changes state in the record to compromised.
 - If the response is "wrong guess", $\mathcal{S}$ passes the response to $\mathcal{A}$ and changes state in the record to interrupted.

On receiving $(\text{RegisterTest}, sid, P)$ from $\mathcal{A}$

- If there is a record $\text{Rec}[P, sid]$ whose state is fresh, $\mathcal{S}$ changes state in the record to interrupted and flag in the record to tested.

Fig. 17. The simulator for Theorem 3 (Part I). The construction of this simulator is mainly based on the simulator for the compiler from lePAKE to rPAKE-EA shown in Fig. 6 of [1,2].

**Fig. 18.** The simulator for Theorem 3 (Part II).

Simulator for Theorem 3 (Part III)	
<u>On receiving (Impersonate, sid, P_C, P_S, uid) from $\mathcal{A}$</u>	
– $\mathcal{S}$ sends (Impersonate, sid, P_C, P_S, uid) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. • If the response is “correct guess”, $\mathcal{S}$ passes the response to $\mathcal{A}$ and changes state in $\text{Rec}[P_C, sid]$ to compromised. • Else, if the response is “wrong guess”, $\mathcal{S}$ passes the response to $\mathcal{A}$ and changes state in $\text{Rec}[P_C, sid]$ to interrupted.	
<u>On receiving a key confirmation message τ^* for P from $\mathcal{A}$</u>	
– If state in the record $\text{Rec}[P, sid]$ is completed, $\mathcal{S}$ proceeds as follows. • If flag in $\text{Rec}[P, sid]$ is tested, $\mathcal{S}$ sends (RegisterTest, sid, P) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. • If key in $\text{Rec}[P, sid]$ is not $\perp$, * If $\tau^* = \gamma$, $\mathcal{S}$ sends (NewKey, $sid, P, \text{PRF}_k(1)$) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. * Else if $\tau^* \neq \gamma$, $\mathcal{S}$ sends (NewKey, $sid, P, \perp$) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. • If key in $\text{Rec}[P, sid]$ is $\perp$, * If state in the record $\text{Rec}[P', sid]$ is completed and its state was fresh before it became completed, and τ^* is τ, which is in $\text{Rec}[P, sid]$, $\mathcal{S}$ sends (NewKey, $sid, P, 0$) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. * Else send (NewKey, $sid, P, \perp$) to $\mathcal{F}_{r\text{-aPAKE-EA}}$.	
<u>On receiving (StealPwdFile, P_S, uid) from $\mathcal{A}$</u>	
– $\mathcal{S}$ sends (StealPwdFile, uid) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. • If the response is pw or “password file stolen”, $\mathcal{S}$ sends the response to $\mathcal{A}$ and changes stateF in $\text{FILE}[P_S, uid]$ to compromised. • Else, if the response is “no password file”, $\mathcal{S}$ sends the response to $\mathcal{A}$.	
<u>On receiving (OfflineTestPwd, uid, pw) from $\mathcal{A}$</u>	
– $\mathcal{S}$ sends (OfflineTestPwd, uid, pw) to $\mathcal{F}_{r\text{-aPAKE-EA}}$. $\mathcal{S}$ passes the response to $\mathcal{A}$.	

Fig. 19. The simulator for Theorem 3 (Part III).